

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

Теория Баз Данных

A solid blue rounded rectangle with a light blue gradient background.

Теория баз данных

Содержание

Unit 1.

Введение в теорию баз данных

1. Введение в теорию баз данных.....	6
История и этапы развития	6
Понятия база данных и система управления базами данных.....	7
Сравнение существующих моделей баз данных	8
Понятие реляционной модели баз данных	12
Двенадцать правил Кодда.....	14
Сравнительный анализ СУБД Microsoft SQL Server с существующими системами управления базами данных	16
2. Основы взаимодействия с Microsoft SQL Server.....	18
Версии и редакции Microsoft SQL Server	18
Инсталляция Microsoft SQL Server.....	18
Инструменты управления и утилиты MS SQL Server ...	32
Управление базой данных	36
Изменение размера базы данных	42
Домашнее задание	48

Unit 2.

Таблицы, типы данных, запросы, основы взаимодействия с MS SQL Server

1. Таблицы.....	50
Первичный ключ.....	52

Значение по умолчанию	56
Уникальность.....	57
2. Типы данных	64
Целочисленные типы данных.....	64
Типы данных для хранения текста	64
Вещественные типы данных	66
Типы данных для хранения даты и времени.....	67
Типы данных с фиксированной точкой	68
Другие типы данных	68
3. Индекс.....	70
Что такое индекс?	70
Цели и задачи индекса.....	70
Внутреннее устройство индекса	71
4. Системные базы данных и таблицы.....	74
5. Запросы	76
Введение в язык структурированных запросов SQL	76
Язык SQL.....	76
Стандарты языка SQL.....	78
Диалекты языка SQL	78
Диалект Transact-SQL	79
Понятия DDL, DML, DCL.....	80
Домашнее задание	81

Unit 1.
Введение
в теорию
баз данных

1. Введение в теорию баз данных

Базы данных используются для хранения и обработки различной информации уже не один десяток лет и будут актуальны еще длительное время. Наверное, вам известна знаменитая фраза Натана Ротшильда: «Кто владеет информацией, тот владеет миром», а где же сейчас хранится эта информация? Правильно, в базах данных, поэтому специалисты в этой области будут востребованы всегда.

История и этапы развития

Исследования по созданию баз данных стали возможны благодаря появлению программируемого оборудования обработки записей и начались в конце 50-х годов прошлого столетия. Основной идеей этих исследований была автоматизация офисной работы, связанная с хранением и учетом данных, которая выполнялась вручную и требовавшая больших затрат труда. Благодаря удешевлению вычислительных мощностей для решения этих задач стало возможным использование компьютеров.

Лидером в сфере исследований баз данных была компания IBM, которая в 1968 году выпустила свою первую систему управления базами данных IMS (*IBM Information Management System*), основанную на иерархической модели данных. В середине 70-х годов прошлого столетия компания IBM создала первую реляционную систему управления базами данных (РСУБД) — IBM

System R, в которой впервые был реализован язык запросов для реляционных баз данных — *Structured Query Language* (SQL).

В 1979 году компания Oracle выпустила первую коммерческую РСУБД Oracle v2, вслед за этим различные компании выпустили свои коммерческие продукты. В 1982 году появилась РСУБД DB2 от IBM, в 1988 году компании Microsoft и Ashton-Tate выпустили совместный продукт Ashton-Tate/Microsoft SQL Server 1.0, в 1992 году появилась первая версия РСУБД Access от компании Microsoft.

Понятия база данных и система управления базами данных

Вы, наверное, встречали в повседневной жизни сочетание слов «база данных», но это довольно широкое понятие, смысл которого сводится к организации хранения данных. Однако различаются способы хранения, например это может быть набор файлов.

В нашем случае, **база данных** (БД) — это коллекции связанных данных, сгруппированных в единый объект. Допустим, при создании базы данных какого-то учебного заведения, мы размещаем в ней необходимые наборы данных (информация о студентах, преподавателях, факультетах и т.д.), и впоследствии управляем полученной БД при помощи программного обеспечения как единым целым.

Специализированное программное обеспечение, позволяющее работать с базами данных (обновлять, извлекать данные и т.д.), имеет общее название система

управления базами данных (СУБД), например Microsoft SQL Server, Access, Oracle Database и т.д.

СУБД состоит из ряда серверных и клиентских средств, которые позволяют выполнять администрирование баз данных и различные действия, связанные с манипуляцией данными. Практически любая СУБД позволяет обрабатывать запросы на извлечение и изменение данных, предоставляет механизмы для резервного копирования и восстановления данных, оптимизирует производительность выполнения запросов, управляет памятью.

Сравнение существующих моделей баз данных

Модели баз данных различаются правилами взаимосвязи основных типов структур данных и операциями над ними. Каждая СУБД использует логическую структуру хранения данных, которая зависит от конкретной модели хранения данных, рассмотрим далее наиболее часто используемые модели баз данных.

Файловая модель

Файловая модель данных характеризуется определенным набором файлов, не связанных между собой, основными типами структур данных которой являются поле, запись, файл. Поле — это элементарная, неделимая единица данных. Запись — это совокупность логически связанных полей. Файл — это множество записей, одинаковых по структуре.

Файловая модель была первой моделью хранения данных, ее можно считать моделью без СУБД, ведь вну-

тренняя структура файлов была известна только разработчику данного программного обеспечения, то есть была уникальна.

Файловая модель обладает рядом недостатков, перечислим основные из них:

- алгоритм управления базой данных полностью заложен в программном обеспечении, при изменении структуры данных необходимо вносить правки в программное обеспечение;
- сложности совместимости форматов файлов, созданных при помощи различных языков программирования, из-за отличающихся друг от друга структур;
- трудности при переносе данных из одной БД в другую, по причине несовпадения структур данных;
- отсутствие централизованного хранения приводит к необходимости дублирования одних и тех же данных.

Иерархическая модель

Иерархическая модель представляет собой простую структуру, в которой отдельные записи организуются в отношения типа «родитель-потомок» и образуют обращенное дерево. Данные в этой иерархической структуре делятся на логические категории и подкатегории, использующие записи для представления логических единиц данных.

Иерархическая модель довольно удобна для представления отношений между сущностями реального мира, однако совершенно не подходит для многих современных приложений. Иерархическая структура не

поддерживает сложные отношения между записями типа «многие ко многим», потому что дочерняя запись может ассоциироваться только с одной родительской записью. Еще одним недостатком является довольно громоздкий процесс навигации, ведь для того чтобы получить доступ к записям необходимо перемещаться вверх и вниз по уровням иерархии.

Наиболее известная СУБД, созданная на основе этой модели — это IMS от компании IBM. Иерархическая модель нашла свое применение не только в СУБД, такая модель применяется для систем управления файлами в операционных системах.

Сетевая модель

Говоря о сетевой модели баз данных нельзя не упомянуть о Чарльзе Бахмане (*Charles Bachman*), который являлся одним из основоположников сетевой модели баз данных. Именно он в 1963 году разработал одну из первых сетевых систем управления базами данных IDS (*Integrated Data Store*). За свои разработки, посвященные технологиям баз данных, в 1973 году он был удостоен премии Тьюринга.

Сетевая модель появилась как результат усовершенствования иерархической модели, и в отличие от последней позволяет записям принимать участие во множестве отношений типа «родитель-потомок».

Сетевая модель была стандартизирована в 1975 году организацией CODASYL (*Conference of Data System Languages*), которая определила базовые понятия этой модели и язык описания.

Хотя сетевая модель является более гибкой по сравнению с иерархической, программирование навигации по записям должны осуществлять разработчики. Реализация этой модели подразумевает использование значительных ресурсов памяти, потому что каждый элемент должен хранить ссылки на другие элементы. Кроме того, внесение любых изменений может привести к сложным операциям обновления базы данных.

Реляционная модель

Еще одним выдающимся ученым был Эдгар Кодд (*Edgar Codd*), будучи сотрудником компании IBM, разработал реляционную модель данных, которую в 1970 году описал в статье «A Relational Model of Data for Large Shared Data Banks». В 1981 году за свои исследования в области баз данных он был награжден премией Тьюринга.

Реляционная модели данных не использует связь между родительскими и дочерними записями, а основана на взаимодействии строк и столбцов, которые образуют таблицы с данными, связанными между собой. Внутри базы данных, таблицы имеют уникальные названия и связываются между собой отношениями, которые позволяют осуществлять навигацию по записям. Такая структура данных является очень гибкой и удобной при извлечении и изменении информации.

В качестве примера РСУБД можно назвать: Microsoft SQL Server, Access, MySQL.

Объектно-ориентированная модель

Объектно-ориентированная модель данных имеет древовидную структуру, узлами которой являются объ-

екты. В этой модели технология объектно-ориентированного программирования (ООП) применяется к технологии баз данных. Каждая запись в базе данных является объектом, связи между записями осуществляются с помощью механизмов, которые используются в ООП. Поиск необходимой записи заключается в сравнении пользовательского объекта с объектами, которые хранятся в БД.

Базовые понятия объектно-ориентированной модели полностью повторяют основные понятия ООП: класс, объект, метод, инкапсуляция, наследование, полиморфизм, агрегация.

Основные недостатки модели объектно-ориентированной модели: сложность понимания ее структуры и относительно низкая скорость выполнения запросов. К достоинствам этой модели можно отнести возможность хранения и отображения информации о сложных объектах, с возможностью определения методов для работы с ними. Благодаря этим достоинствам некоторые РСУБД дополняются элементами объектно-ориентированного проектирования, например Oracle Database, которая является объектно-реляционной СУБД.

Понятие реляционной модели баз данных

Начиная с конца 70-х годов прошлого столетия наиболее популярной моделью баз данных, является реляционная модель, в основе которой лежат математические принципы обработки данных.

В реляционной модели данные представлены в виде таблиц, которые состоят из полей. Например, на рисун-

ке 1.1 показаны две таблицы, в первой из них хранится информация о студентах, а во второй о группах, в которых проходят обучение студенты.

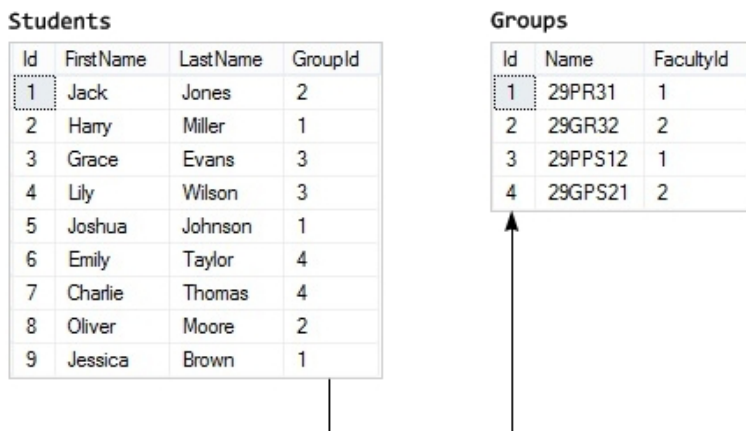


Рисунок 1.1. Пример структуры реляционной базы данных

Каждое поле предназначено для хранения определенной информации, которая характеризует конкретную сущность. В нашем случае каждая запись в таблице **Students** представляет конкретного студента, в которой указывается уникальный идентификатор студента, имя, фамилия и идентификатор той группы, где он обучается. В таблице **Groups** записи содержат уникальную информацию по каждой группе, и состоят из идентификатора группы, имени группы и идентификатора факультета.

В литературе, описывающей реляционную модель данных, вы можете встретить несколько другие определения: так таблицы называются отношениями (*англ. relation*), в общем, отсюда и пошло название самой модели — реляционная, запись в таблице называются кортежами, а поля — атрибутами. Однако какие бы вы названия

не использовали, суть реляционной модели останется прежней.

Как показано на рисунке 1.1 связь между таблицами обеспечивается благодаря наличию в одной таблице уникального идентификатора из другой таблицы, мы сейчас не будем останавливаться на этом подробно, более полную информацию вы получите в последующих уроках.

Двенадцать правил Кодда

В процессе разработки реляционной модели баз данных, Эдгар Кодд сформулировал требования, которым должна соответствовать любая реляционная СУБД, он опубликовал их в 1985 году. На самом деле основное правило имеет номер 0, поэтому всего насчитывается 13 правил. Перечислим эти правила:

1. Основное правило: СУБД должна управлять базами данных, используя исключительно свои реляционные возможности;
2. Правило информации: вся информация в любой базе данных должна быть представлена исключительно значениями в таблицах;
3. Правило гарантированного доступа: каждое значение в любой таблице можно получить с помощью комбинации имени таблицы, значения первичного ключа и имени столбца (имя таблицы позволяет найти требуемую таблицу, имя столбца позволяет найти требуемый столбец, а первичный ключ позволяет найти строку, содержащую искомый элемент данных);
4. Правило поддержки недействительных значений: в СУБД должна быть реализована возможность под-

- держки неизвестных или отсутствующих значений (отсутствующие данные должны быть представлены с помощью недействительных значений (NULL));
5. Правило динамического каталога: информация о базах данных должна храниться в виде таблиц, и СУБД должна обеспечивать стандартный доступ к ней и пользовательским данным при помощи одних и тех же средств (СУБД должна содержать набор системных таблиц, описывающих структуру баз данных);
 6. Правило исчерпывающего подязыка данных: СУБД должна поддерживать хотя бы один язык, операторы которого обеспечивают все ее основные функции (создание базы данных, манипулирование данными, управление доступом и т.д.);
 7. Правило обновления представлений: каждое представление должно поддерживать те же операции с данными, что и таблицы (чтение, вставка, изменение и удаление данных);
 8. Правило добавления, обновления и удаления: операции связанные с изменением и удалением данных должны одинаково работать как с одной записью в таблице, так и с множеством записей;
 9. Правило независимости физических данных: приложения, использующие любую базу данных, не должны зависеть от способа хранения информации и аппаратного обеспечения компьютеров;
 10. Правило независимости логических данных: приложения не должны зависеть от структуры базы данных, любые изменения в структуре БД не должны влиять на работу приложения;

11. Правило независимости условий целостности: язык СУБД должен поддерживать проверку входных данных и обеспечивать их целостность;
12. Правило независимости распространения: база данных может находиться на разных компьютерах и язык СУБД должен поддерживать возможность работы с распределенными данными;
13. Правило единственности: не должно быть возможности нарушить безопасность и целостность данных в обход языка СУБД, то есть при работе с данными должен использоваться только язык СУБД.

Сравнительный анализ СУБД Microsoft SQL Server с существующими системами управления базами данных

На сегодняшний день существует большое количество систем управления базами данных, некоторые из них уже упоминались нами в текущем уроке. В качестве сравниваемых СУБД мы возьмем MS SQL Server, Oracle, PostgreSQL и MySQL, у каждой из них есть как свои преимущества, так и недостатки. Сейчас вы, наверное, скажете: «Всяк кулик своё болото хвалит», однако мы все же попытаемся быть максимально объективными.

Если рассматривать Oracle и MS SQL Server, то по производственным характеристикам они приблизительно одинаковы, может быть по ряду показателей Oracle лучше. Главное же отличие Oracle от MS SQL Server — это поддержка большого количества программно-аппаратных платформ (Linux, Windows, Mac OS и т.д.). В качестве отрицательных сторон Oracle можно назвать: высокую стоимость, высокие требования к аппаратному обеспече-

нию и сложность администрирования. К тому же Oracle в основном предназначен для использования в больших промышленных проектах, и в относительно простых приложениях его использование будет не рационально.

MySQL — это РСУБД, предназначенная для использования в простых и средних приложениях и основное ее преимущество в том, что она бесплатная. К недостаткам можно отнести: ограниченную функциональность и некоторые проблемы с надежностью.

PostgreSQL — это свободно распространяемая объектно-реляционная СУБД, максимально соответствующая стандартам SQL, поддерживаемая ОС Windows и множеством UNIX-подобных платформ. И хотя PostgreSQL является довольно качественным программным продуктом, у нее тоже есть недостатки: производительность, сложность настройки и небольшая популярность (малое количество хостингов с поддержкой этой СУБД).

Из всего вышеперечисленного можно сделать вывод, что MS SQL Server является некой золотой серединой среди СУБД, хотя, конечно же, у нее есть и свои недостатки.

При изучении текущего предмета мы будем использовать СУБД Microsoft SQL Server, поэтому далее остановимся на ней более подробно.

2. Основы взаимодействия с Microsoft SQL Server

Версии и редакции Microsoft SQL Server

Как мы уже говорили ранее, первая версия MS SQL Server (SQL Server 1.0) появилась в 1988 году. Следующая версия — SQL Server 1.1 — вышла в 1990 году. За прошедшие двадцать с лишним лет вышло 15 версий MS SQL Server.

В 2016 году вышла последняя, на текущий момент времени, версия — SQL Server 2016, именно эту версию СУБД MS SQL Server мы и будем использовать при изучении текущего предмета, но прежде чем использовать тот или иной программный продукт его необходимо установить на компьютер.

Инсталляция Microsoft SQL Server

СУБД Microsoft SQL Server 2016 доступна в нескольких редакциях: Enterprise, Standard, Developer и Express. Для изучения материалов курса вам достаточно установить версию MS SQL Server 2016 Express, к тому же она бесплатная.

Прежде всего, вам необходимо скачать установщик программного продукта, для этого нужно перейти на сайт компании Microsoft по данному адресу: <https://www.microsoft.com/en-us/download/details.aspx?id=54284>. На этой же странице необходимо ознакомиться с системными требованиями, которым должен соответствовать

ваш компьютер, для того чтобы установка MS SQL Server была успешной. Перечислим эти требования:

- поддерживаемые операционные системы: Windows 10, Windows 8, Windows 8.1, Windows Server 2012, Windows Server 2012 R2, Windows Server 2016;
- минимальная скорость процессора: 1 ГГц;
- минимальный объем оперативной памяти: 512 МБ;
- свободное место на жестком диске: 4,2 ГБ.

После скачивания и запуска установщика, появится окно, позволяющее выбрать тип установки. Если выбрать базовый тип установки, то ядро MS SQL Server будет сконфигурировано по умолчанию. Мы же выберем пользовательский тип установки — это расширенный тип установки, требующий больше времени (Рисунок 2.1).

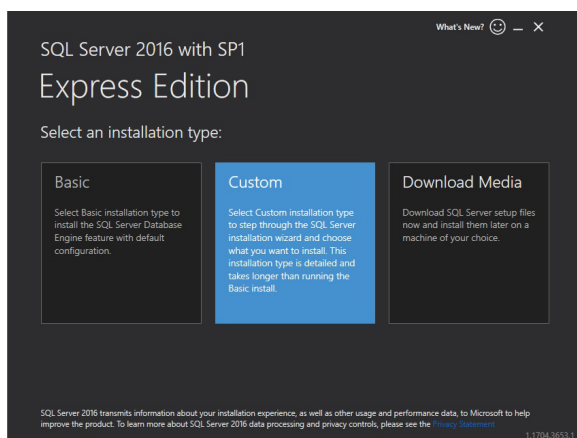


Рисунок 2.1. Выбор пользовательского типа установки

В следующем окне отображается необходимое свободное пространство на диске, размер скачивания и папка на компьютере, куда будет производиться установка,

в случае необходимости ее можно изменить. Оставим расположение установки по умолчанию и запустим установку, нажав кнопку **Install** (Рисунок 2.2).

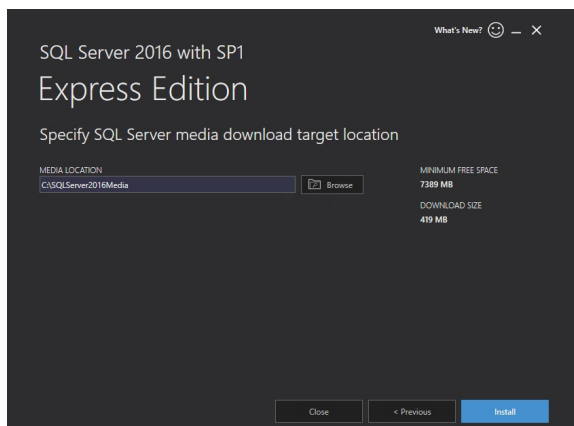


Рисунок 2.2. Начало установки

После этого начинается скачивание пакета установки, процесс скачивания отображается при помощи индикатора (Рисунок 2.3).

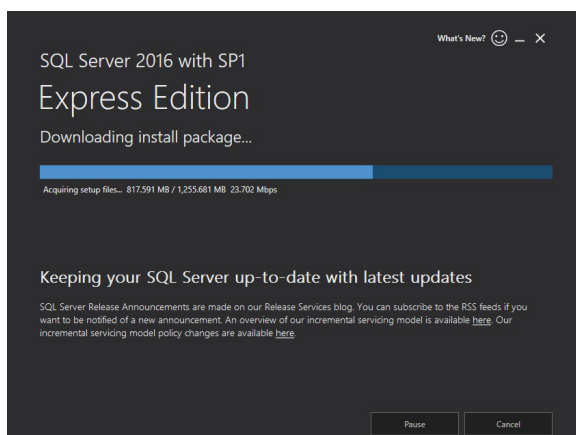


Рисунок 2.3. Скачивание пакета установки

После того как будет скачан пакет установки, появиться окно установки MS SQL Server 2016 с выбранным пунктом **Installation**. В списке возможных установок необходимо выбрать первый пункт — **New SQL Server stand-alone installation...** (Рисунок 2.4).

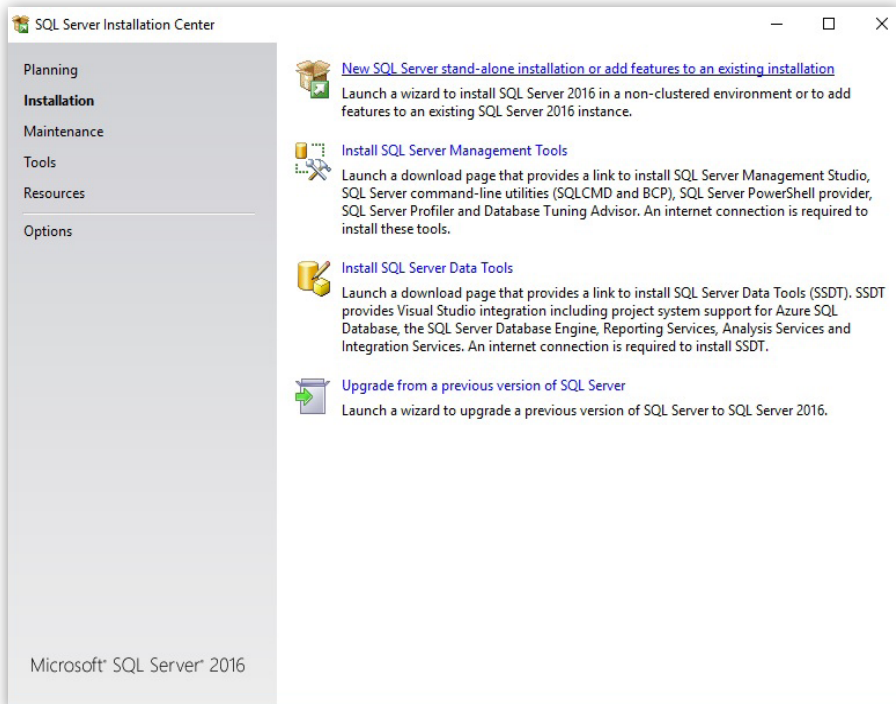


Рисунок 2.4. Начало новой установки экземпляра MS SQL Server

В следующем окне предлагается ознакомиться с лицензионным соглашением от корпорации Microsoft по использованию Microsoft SQL Server 2016 Express. После внимательного ознакомления, его следует принять и нажать кнопку **Next** (Рисунок 2.5).

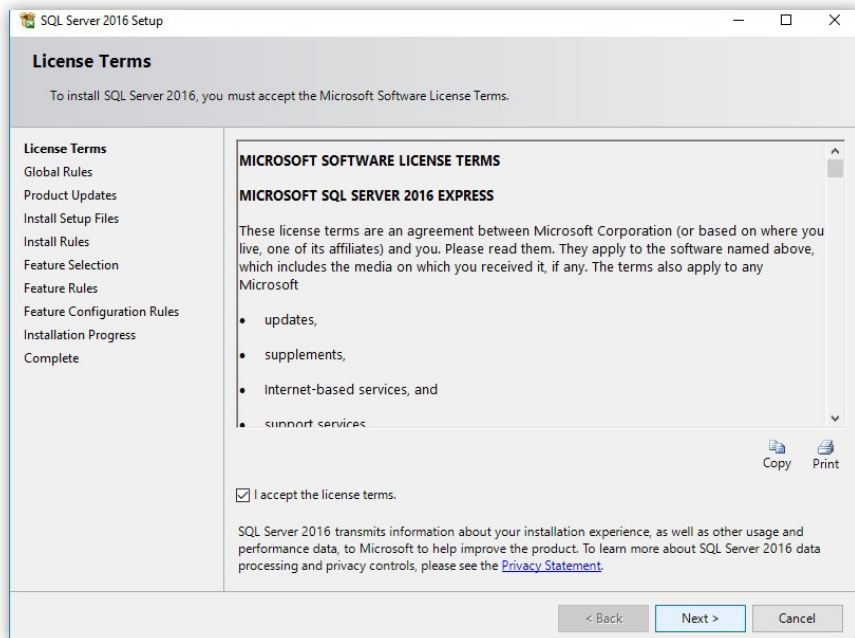


Рисунок 2.5. Лицензионное соглашение от корпорации Microsoft

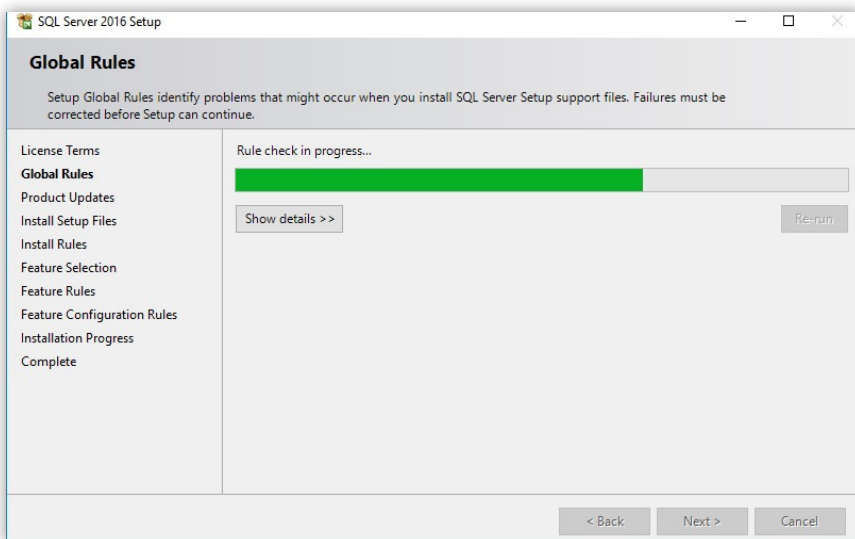


Рисунок 2.6. Установка файлов

После этого осуществляется поиск обновлений, загрузка и установка файлов установки (Рисунок 2.6).

Затем программа установки определяет потенциальные проблемы, которые могут возникнуть при выполнении установки MS SQL Server. В нашем случае появилось предупреждение, в котором говорится о необходимости открытия соответствующих портов для обеспечения удаленного доступа. Мы учтем это предупреждение при настройке удаленного доступа, а пока что нажмем кнопку **Next** (Рисунок 2.7).

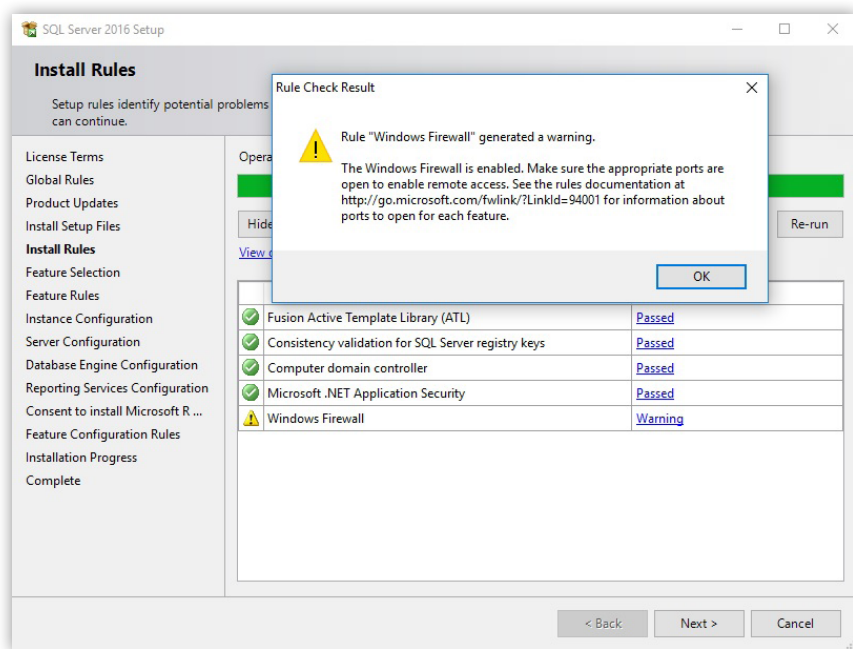


Рисунок 2.7. Правила установки

В следующем окне установки необходимо выбрать компоненты для установки и перейти на следующее окно (Рисунок 2.8).

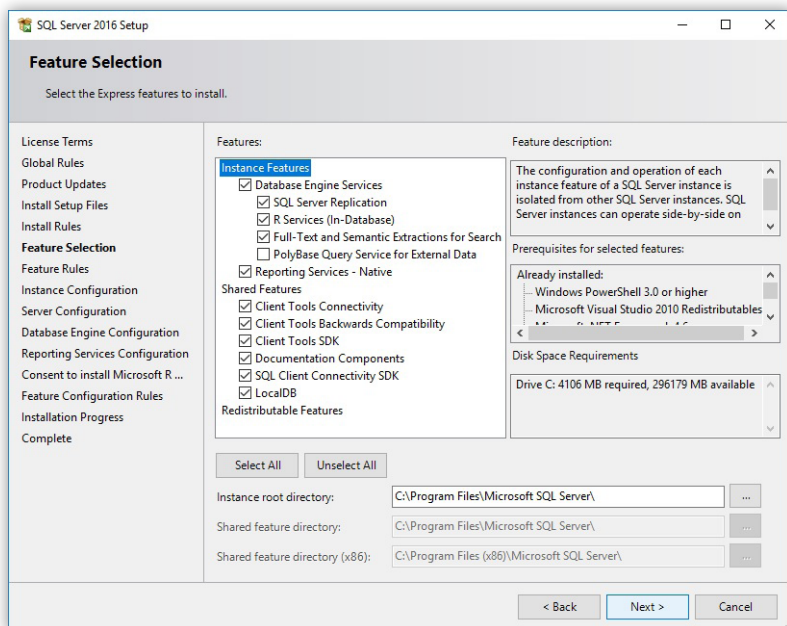


Рисунок 2.8. Выбор компонентов

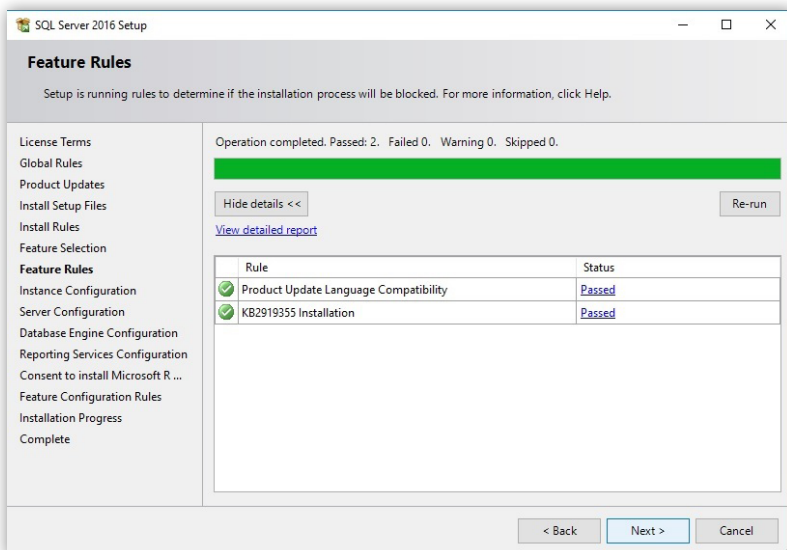


Рисунок 2.9. Установка компонентов

В следующем окне демонстрируется процесс и результат установки выбранных ранее компонентов, убедившись, что установка прошла успешно нужно нажать кнопку **Next** (Рисунок 2.9).

Далее вы увидите окно, в котором необходимо указать имя и идентификатор для экземпляра SQL Server, выберем пункт **Named instance** (*Именованный экземпляр*) и перейдем далее (Рисунок 2.10).

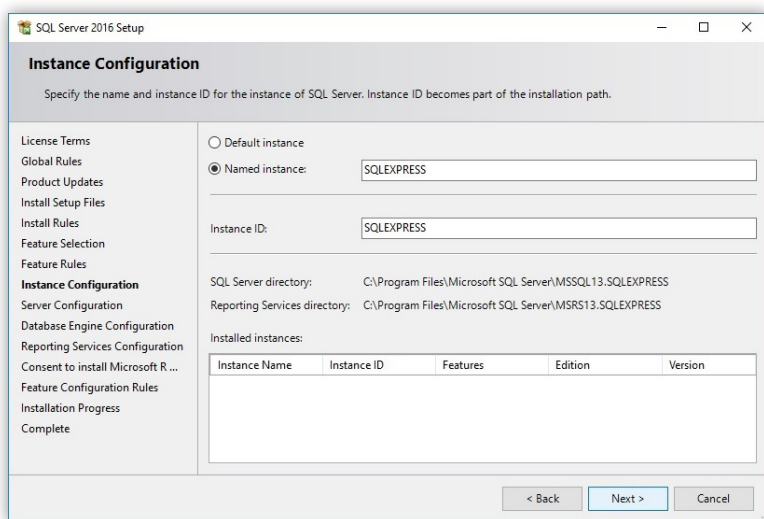


Рисунок 2.10. Настройка экземпляра

Следующее окно предназначено для настройки учетных записей и параметров сортировки, оставим эти пункты без изменений и нажмем кнопку **Next** (Рисунок 2.11).

Следующее окно позволяет настроить компонент **Database Engine**. На вкладке **Server Configuration** нужно выбрать один из двух режимов проверки подлинности, либо **Windows authentication mode** (*режим проверки подлинности Windows*), либо **Mixed Mode** (*смешанный ре-*

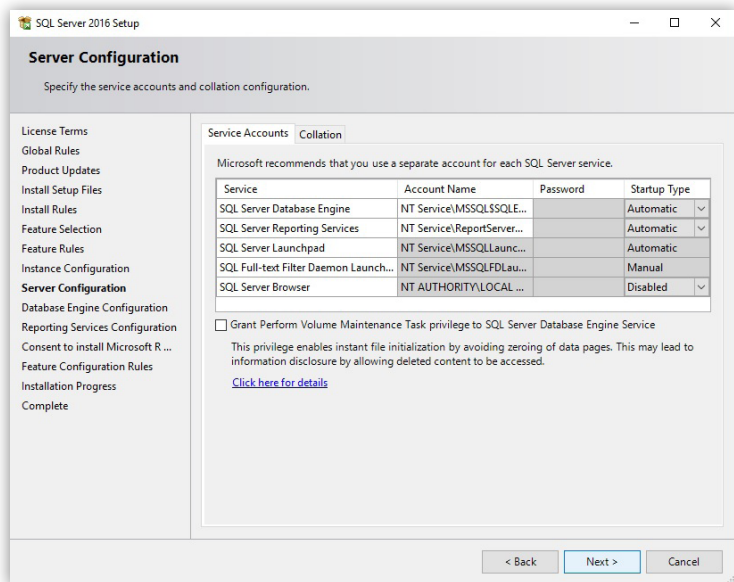


Рисунок 2.11. Конфигурация сервера

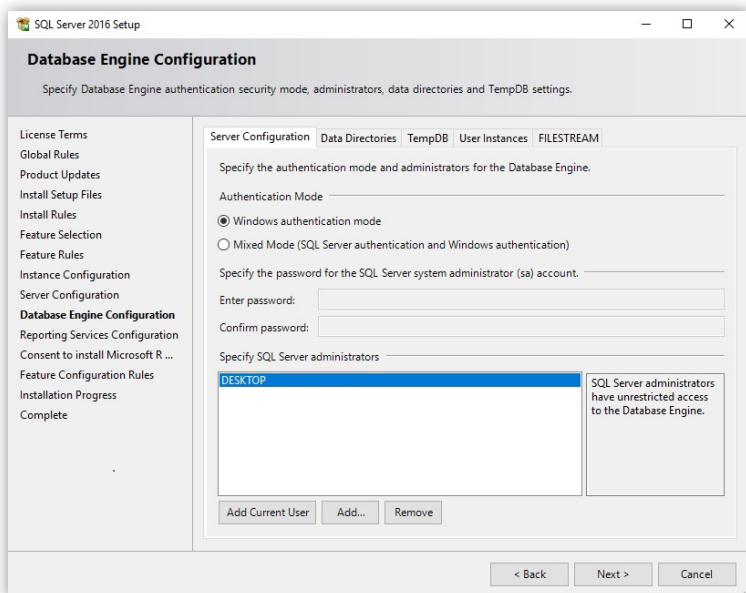


Рисунок 2.12. Вкладка «Конфигурация сервера»

жим), при выборе которого необходимо указать пароль для встроенной учетной записи администратора SQL Server (sa), оставим здесь значение по умолчанию. Также необходимо указать администраторов SQL Server, по умолчанию в списке будет находиться имя текущего пользователя. При помощи соответствующих кнопок вы можете, как добавить пользователя или текущего пользователя, так и удалить пользователя из списка (Рисунок 2.12).

На вкладке **Data Directories** задим корневой каталог данных. Можно оставить значение по умолчанию, однако рекомендуется указывать каталог на другом физическом диске, в крайнем случае, на другом логическом диске, что повышает вероятность сохранения данных в случае отказа системы. В нашем случае, за неимением еще одного физического диска, укажем заранее созданный каталог SQLDB на логическом диске D, для чего нажмем кнопку с тремя точками в пункте **Data root directory** (Рисунок 2.13).

После нажатия кнопки **OK** в окне **Browse For Folder** (Рисунок 2.13), пути ко всем каталогам данных обновятся автоматически. Можно изменить расположение каталогов пользовательской базы данных и каталога резервного копирования, проделав аналогичные операции, но мы остановимся на полученном результате (Рисунок 2.14).

Вкладка **TempDB** предназначена для настройки базы данных, хранящей временные данные, на следующей вкладке **User Interfaces** существует возможность установить разрешение обычным пользователям запускать экземпляр компонента **Database Engine** (по умолчанию установлено) и последняя вкладка позволяет включить **FILESTREAM** для интеграции Database Engine с файловой

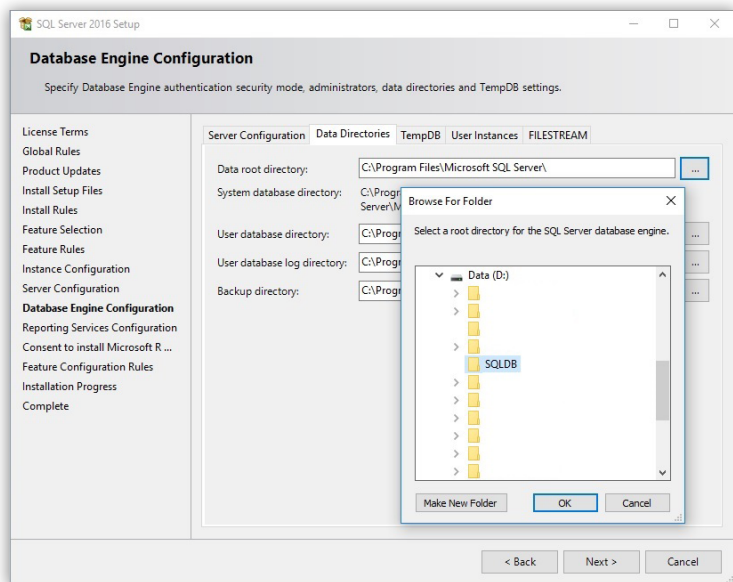


Рисунок 2.13. Установка корневого каталога данных

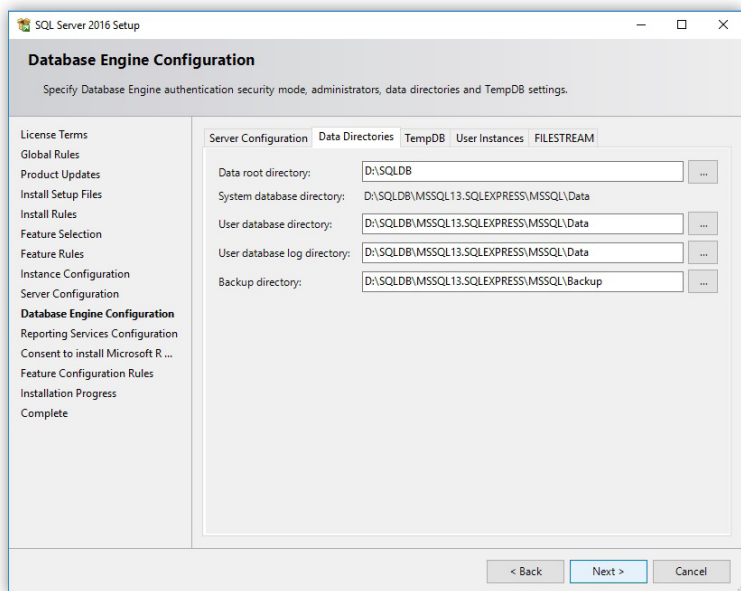


Рисунок 2.14. Вкладка «Каталоги данных»

системой NTFS (по умолчанию отключено). Оставим эти три вкладки без изменений и перейдем далее.

Следующее окно позволяет задать настройку служб **Reporting Services**, выберем пункт **Install and configure** и нажмем кнопку **Next** (Рисунок 2.15).

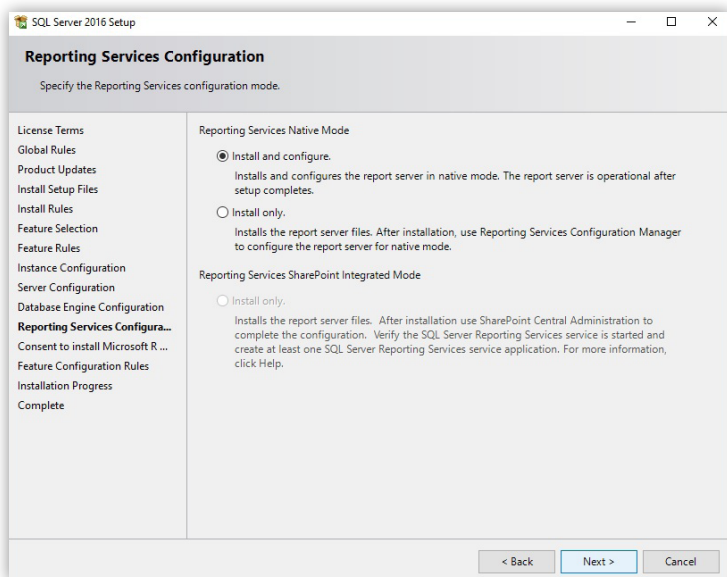


Рисунок 2.15. Настройка служб Reporting Services

Для продолжения установки SQL Server, в следующем окне необходимо нажать кнопку **Accept**, чтобы установить на ваш компьютер расширенное распределение R от корпорации Microsoft — Microsoft R Open, которое поддерживает интеллектуальное моделирование и работу со статистическими данными (Рисунок 2.16).

Нажав кнопку **Next**, после успешной установки Microsoft R Open начнется процесс установки SQL Server на ваш компьютер, ход выполнения которой вы увидите в следующем окне (Рисунок 2.17).

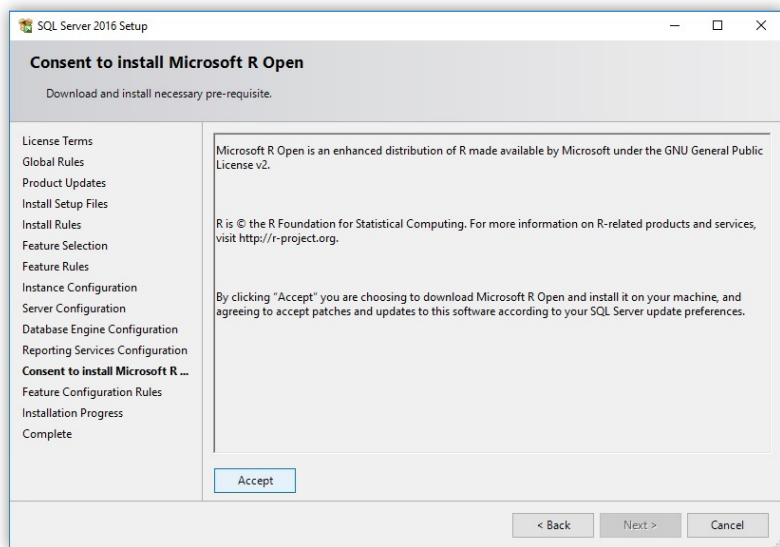


Рисунок 2.16. Установка Microsoft R Open

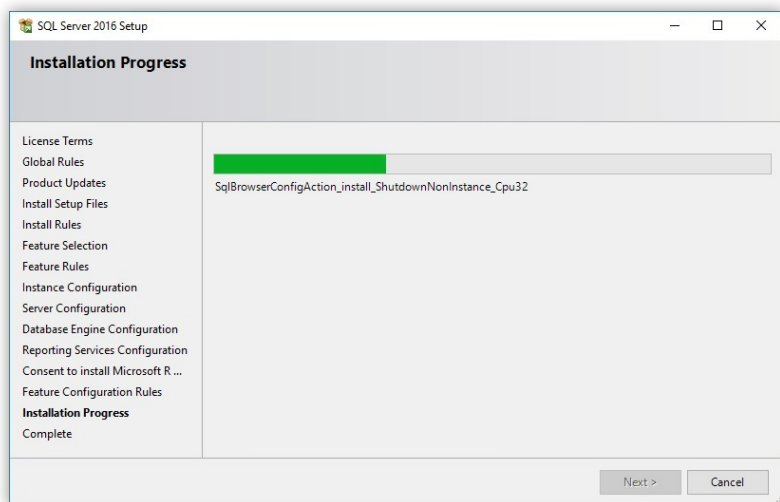


Рисунок 2.17. Ход выполнения установки

По прошествии определенного времени, процесс установки завершится, и вы увидите окно завершения

установки SQL Server, которое содержит сведения об установленных компонентах (Рисунок 2.18).

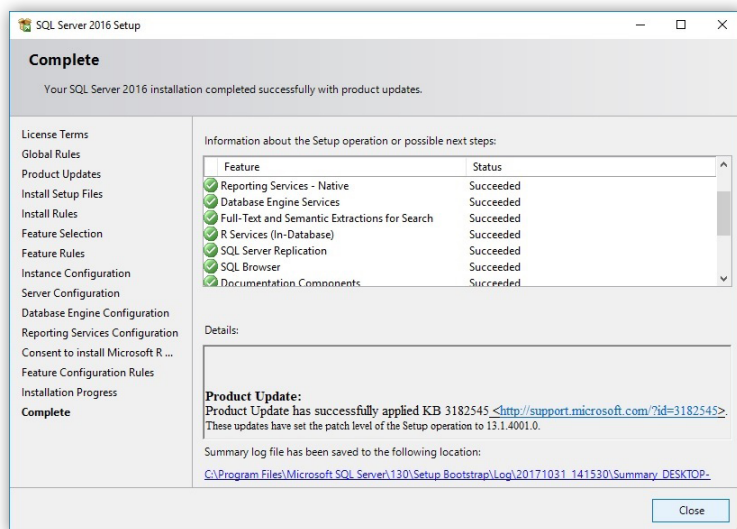


Рисунок 2.18. Завершение установки

По завершению процесса установки, откроем список установленных программ и убедимся, что SQL Server 2016 установлен на ваш компьютер (Рисунок 2.19).

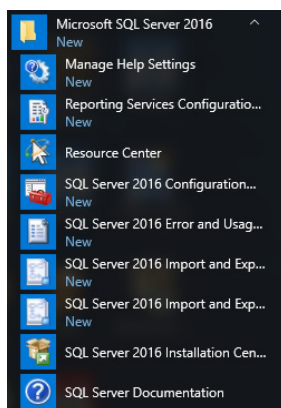


Рисунок 2.19. Программа Microsoft SQL Server 2016

Инструменты управления и утилиты MS SQL Server

В списке установленных компонентов Microsoft SQL Server 2016 вы заметили ряд утилит, кратко остановимся на каждой из них.

Утилита **Reporting Services Configuration Manager** используется для настройки сервера отчетов: учетной записи, базы данных, электронной почты и т.д.

Компонент **SQL Server 2016 Configuration Manager** предназначен для управления службами, связанными с SQL Server (запуск, приостановка, возобновление и остановка служб). При помощи этой утилиты осуществляется управление конфигурацией сетевых подключений с клиентских компьютеров и настройки сетевых протоколов, используемых SQL Server.

Для того чтобы улучшить функциональность SQL Server, корпорация Microsoft использует отчеты об ошибках, за сбор и отправку этой информации отвечает утилита **SQL Server 2016 Error and Usage Reporting**.

Утилита **SQL Server 2016 Import and Export Data** обеспечивает удобный способ копирования данных между источниками данных.

В списке установленных компонентов SQL Server 2016 отсутствует один очень важный инструмент управления — **Microsoft SQL Server Management Studio**, именно этот компонент обеспечивает наличие визуальной среды для работы с SQL Server. Следующее наши действия будут направлены на установку этого компонента, но перед этим необходимо перезагрузить ваш компьютер.

Для установки Management Studio в списке компонентов SQL Server 2016 (Рисунок 2.19) необходимо выбрать

утилиту **SQL Server 2016 Installation Center**, которая обеспечивает установку необходимых компонентов в SQL Server, нажав соответствующий пункт. После этого откроется одноименное окно (Рисунок 2.20).

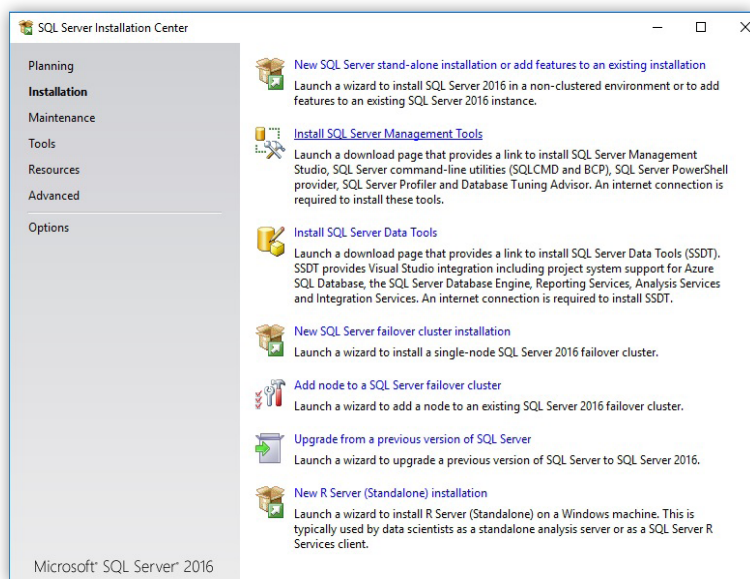


Рисунок 2.20. Центр установки SQL Server

В открывшемся окне необходимо выбрать пункт **Install SQL Server Management Tools**, после чего в окне браузера откроется страница по следующему пути <https://docs.microsoft.com/en-us/sql/ssms/download-sql-server-management-studio-ssms>, на этой странице необходимо выбрать пункт **Download SQL Server Management Studio 17.3**, после чего начнется загрузка установщика Management Studio на ваш компьютер.

После запуска установщика, появится начальное окно установки, чтобы начать установку нужно нажать кнопку **Install** (Рисунок 2.21).

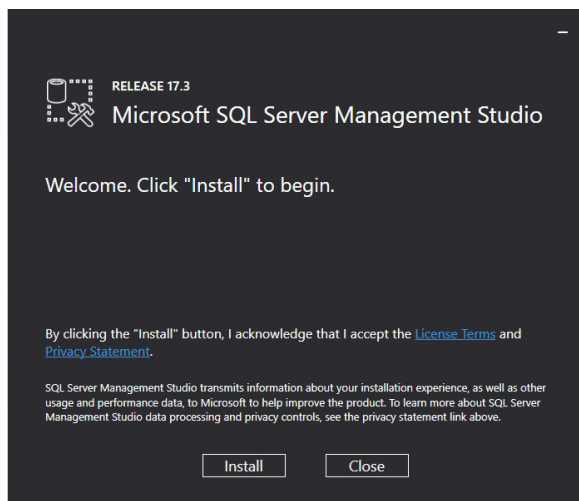


Рисунок 2.21. Начало установки SQL Server Management Studio

Первоначально появится окно загрузки пакетов установки (Рисунок 2.22).

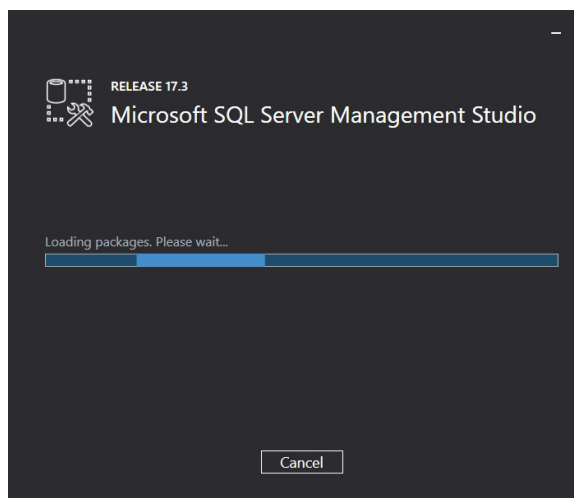


Рисунок 2.22. Загрузка пакетов установки

После того как пакеты будут загружены начнется сам процесс установки SQL Server Management Studio (Рисунок 2.23).

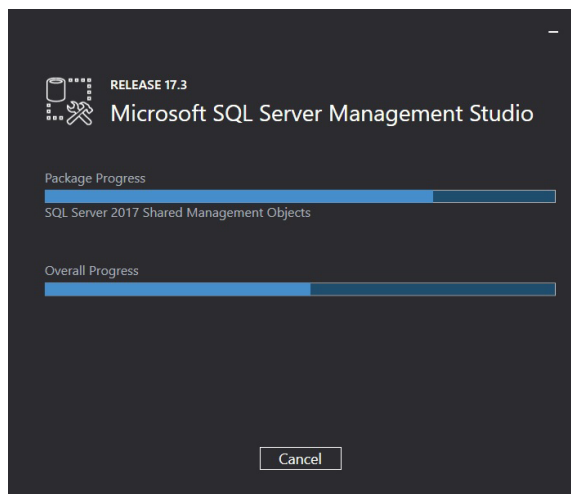


Рисунок 2.23. Процесс установки SQL Server Management Studio

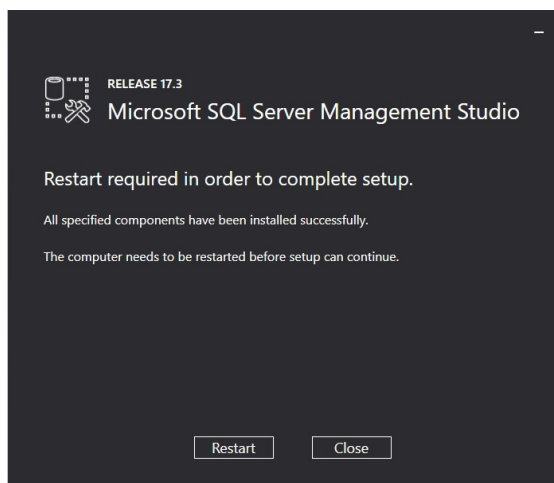


Рисунок 2.24. Окончание установки SQL Server Management Studio

По окончании установки SQL Server Management Studio появится окно, в котором предлагается перезагрузить ваш компьютер, для чего необходимо нажать кнопку **Restart** (Рисунок 2.24).

После перезагрузки компьютера в списке установленных программ появится пункт **Microsoft SQL Server Tools 17**, раскрыв который вы увидите пункт **Microsoft SQL Server Management Studio 17** (Рисунок 2.25).



Рисунок 2.25. Microsoft SQL Server Management Studio

Управление базой данных

Чтобы создать базу данных, запускаем Management Studio, нажав соответствующий пункт (Рисунок 2.25).

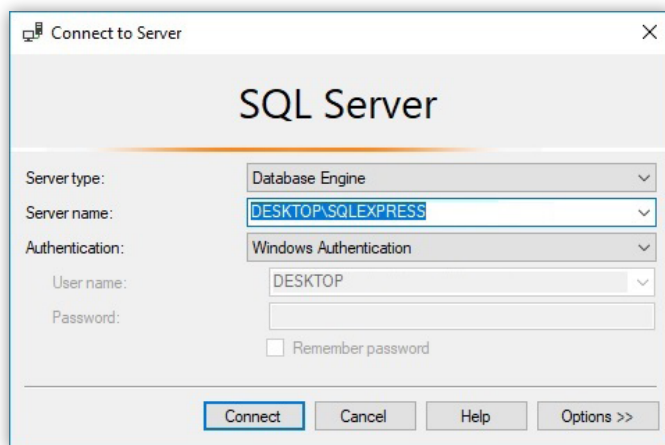


Рисунок 2.26. Подключение к серверу

При первом запуске вы увидите окно загрузки пользовательских настроек, после чего запустится оболочка и появится окно подключения к серверу (Рисунок 2.26).

В открывшемся окне можно сделать несколько дополнительных настроек, кратко остановимся на каждой из них.

При помощи первой настройки (*Server type*) можно задать одну из технологий управления и анализа данных:

- **Database Engine** — основная служба, обеспечивающая безопасность данных, их хранение и обработку (значение по умолчанию);
- **Analysis Services** — служба, предназначенная для обработки аналитических данных и включает набор средств для бизнес-аналитики;
- **Reporting Services** — служба, предоставляющая средства для создания различного рода отчетов, с возможностью их публикации в разнообразных форматах;
- **Integration Services** — служба, которая включает различного рода решения по интеграции данных (извлечение, преобразование, загрузка) (Рисунок 2.27).

Имя сервера (*Server name*) прописывается автоматически, но в случаях, когда требуется указать его явно, необходимо открыть выпадающий список и выбрать требуемый сервер (Рисунок 2.28).

При помощи последней настройки (*Authentication*) можно задать режим проверки подлинности пользователя:

- **Windows Authentication** — этот режим задан по умолчанию и позволяет пользователю при подключении использовать учетную запись ОС Windows;

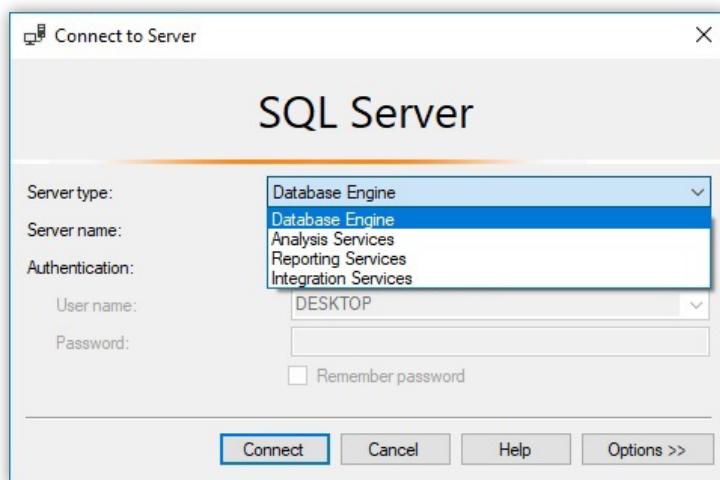


Рисунок 2.27. Технологии управления и анализа данных

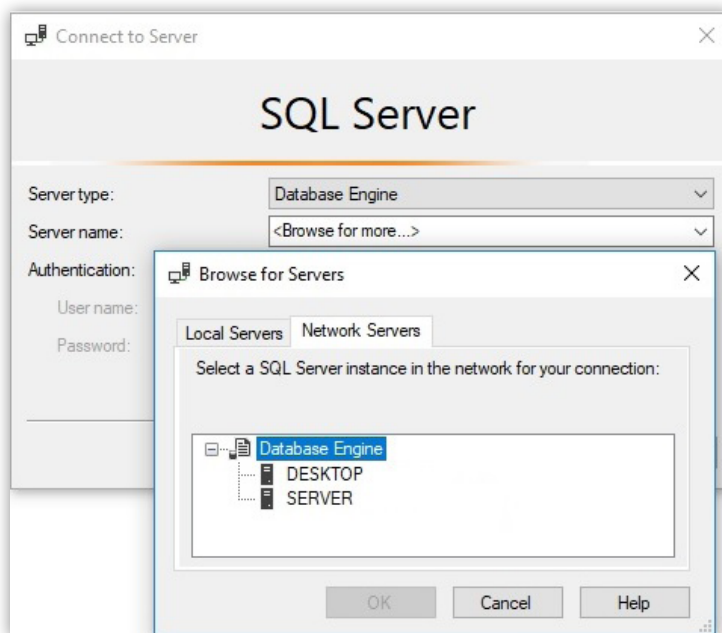


Рисунок 2.28. Выбор сервера

- **SQL Server Authentication** — режим, при котором подлинность пользователя определяется средствами SQL Server, в этом случае проверяется соответствие имени пользователя и пароля;
- **Active Directory – Universal with MFA support** — режим, который обеспечивает аутентификацию пользователя при помощи различных вариантов проверки (телефонный звонок, текстовое сообщение и т.д.);
- **Active Directory – Password** — режим аутентификации, который позволяет подключиться к базе данных Microsoft Azure SQL, используя идентификаторы в Azure Active Directory, если текущая учетная запись не связана с Azure;
- **Active Directory – Integrated** — режим аутентификации, который подобен предыдущему, но применяется, если при входе в ОС Windows использовалась учетная запись Azure Active Directory (Рисунок 2.29).

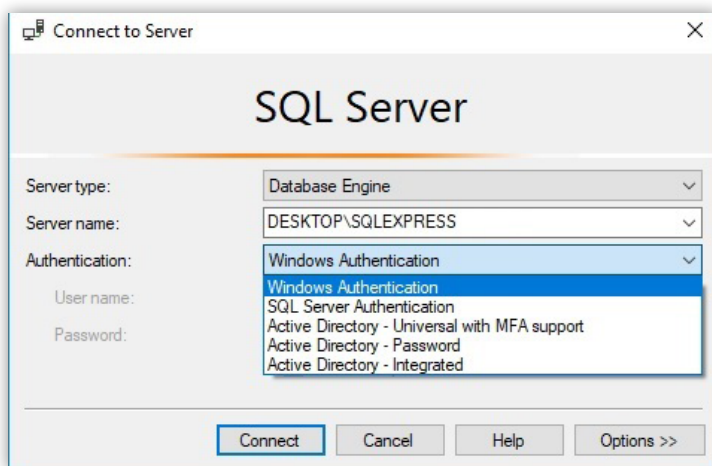


Рисунок 2.29. Выбор режима аутентификации

Оставьте все настройки по умолчанию (Рисунок 2.26) и нажмите кнопку **Connect**. После этого откроется Management Studio, в левой части которой вы увидите окно **Object Explorer**. В этом окне находится список папок, в которых содержатся различные настройки SQL Server, сейчас нас интересует папка **Databases**, с остальными папками ознакомимся по мере необходимости (Рисунок 2.30).

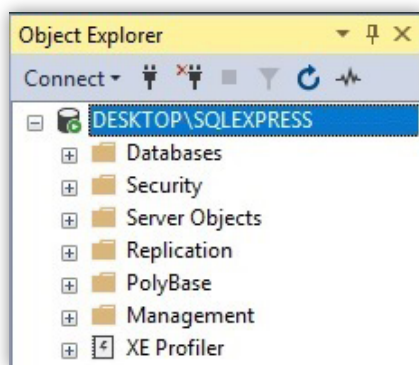


Рисунок 2.30. Окно Object Explorer

Создание базы данных

Для того чтобы создать новую базу данных необходимо на папке **Databases** нажать правой клавишей мыши и в контекстном меню выбрать пункт **New Database...** (Рисунок 2.31).

После этих действий появится окно **New Database**, в котором необходимо указать имя базы данных, в нашем случае **University**, при этом автоматически изменяются логические имена файлов БД (*таблица Database files*). Также можно указать владельца базы данных, здесь мы оставим значение по умолчанию (*default*) (Рисунок 2.32).

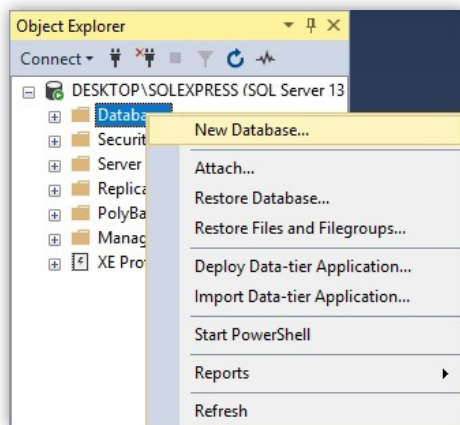


Рисунок 2.31. Начало создания новой базы данных

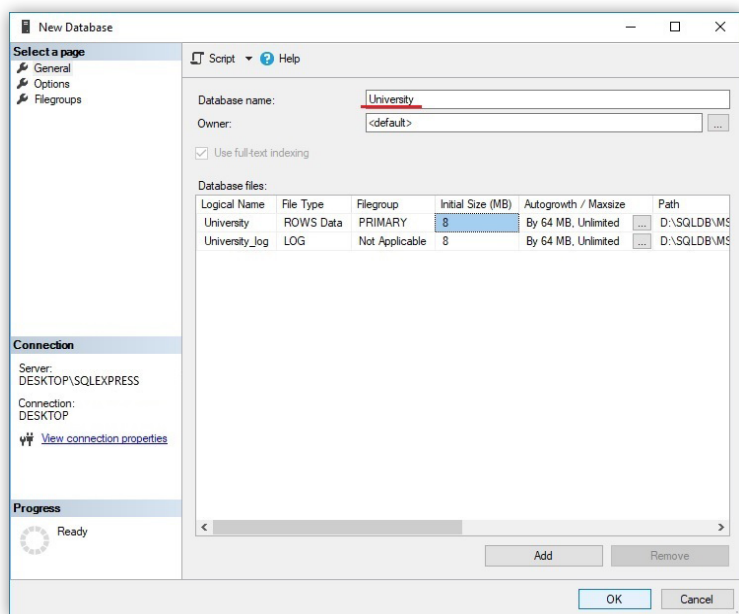


Рисунок 2.32. Имя новой базы данных

Хотя для создания базы данных изменять остальные настройки не обязательно, мы остановимся на них более подробно.

Настройка параметров базы данных

Любая база данных в Microsoft SQL Server состоит из двух физических файлов. Файл с расширением **.mdf** — это основной файл данных, в нем содержится информация обо всех объектах БД. Файл с расширением **.ldf** — это файл журнала транзакций, он играет роль некоего предварительного хранилища данных. Параметры этих файлов находятся в таблице [Database files](#).

Изменение размера базы данных

Чтобы изменить размер базы данных изменяем три параметра: начальный размер, автоматическое приращение размеров и максимальный размер файлов БД.

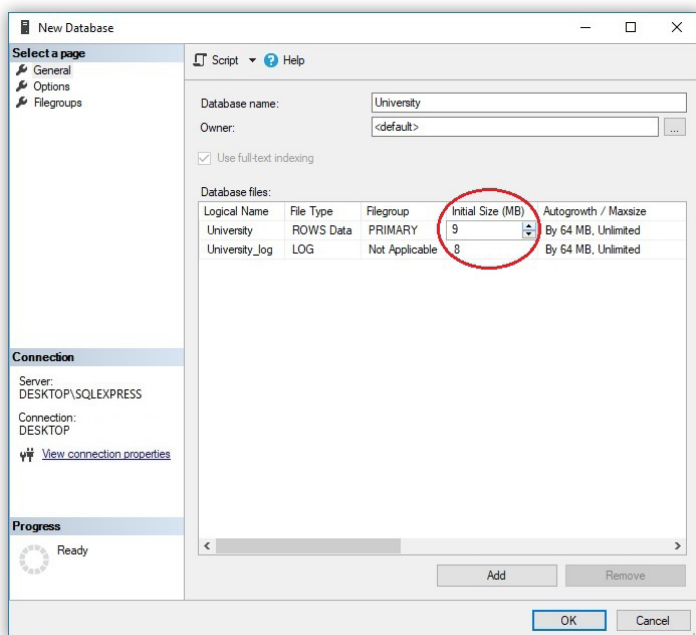


Рисунок 2.33. Изменение начального размера файлов базы данных

Начальный размер файлов базы данных изменяется указанием целочисленного значения размера в поле **Initial Size** таблицы **Database files** (Рисунок 2.33).

Максимальный размер и автоматическое приращение размера конкретного файла базы данных можно настроить, нажав кнопку с тремя точками в соответствующей ячейке поля **Autogrowth/Maxsize**, после чего появиться окно **Change Autogrowth for University** (Рисунок 2.34).

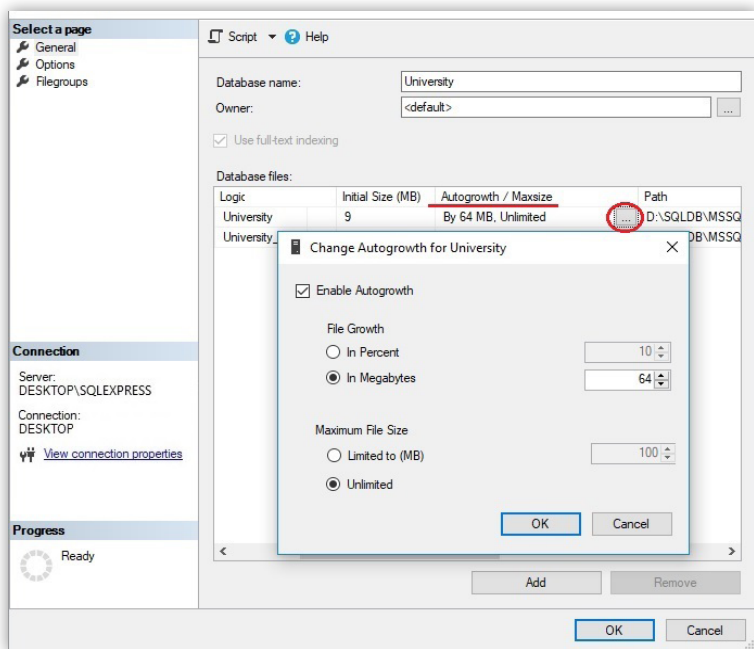


Рисунок 2.34. Изменение максимального размера и приращения размера файла БД

В окне **Change Autogrowth for University** вы можете задать автоматическое приращение размера файла либо в процентах, либо в мегабайтах, указав необходимое значение. Аналогичным образом можно задать макси-

мальный размер файла базы данных, либо указав размер в мегабайтах, либо выбрав пункт **Unlimited**.

Управление группами файлов

Файлы данных базы данных подразделяются на файловые группы, которые используются для объединения файлов с целью повышения производительности. В любой базе данных существует файловая группа **PRIMARY**, в ней находится первичный файл данных. Также существует возможность создания пользовательских файловых групп, для этого необходимо в списке **Select a page** выбрать пункт **Filegroups**, после чего появится соответствующий интерфейс (Рисунок 2.35).

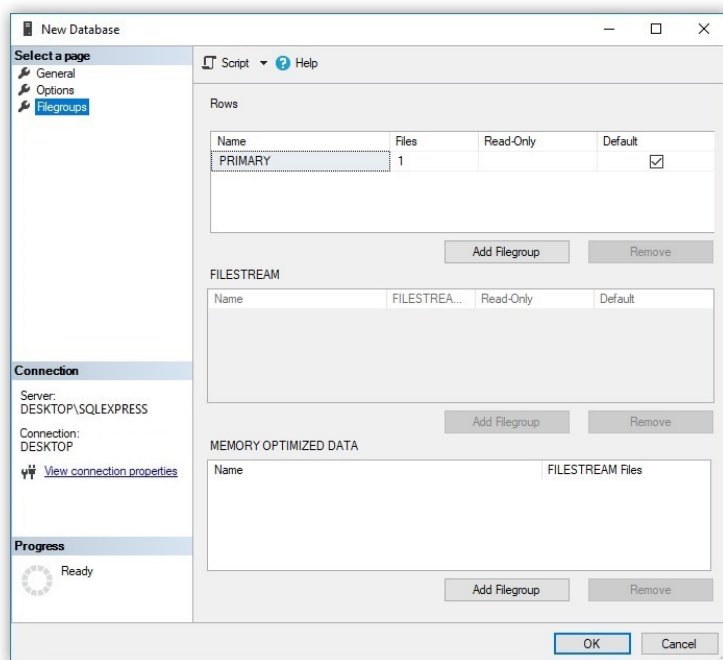


Рисунок 2.35. Управление группами файлов

После того как вы сделаете все настройки вашей базы данных и нажмете кнопку **ОК** в окне **New Database**, вы увидите индикатор создания БД (Рисунок 2.36).

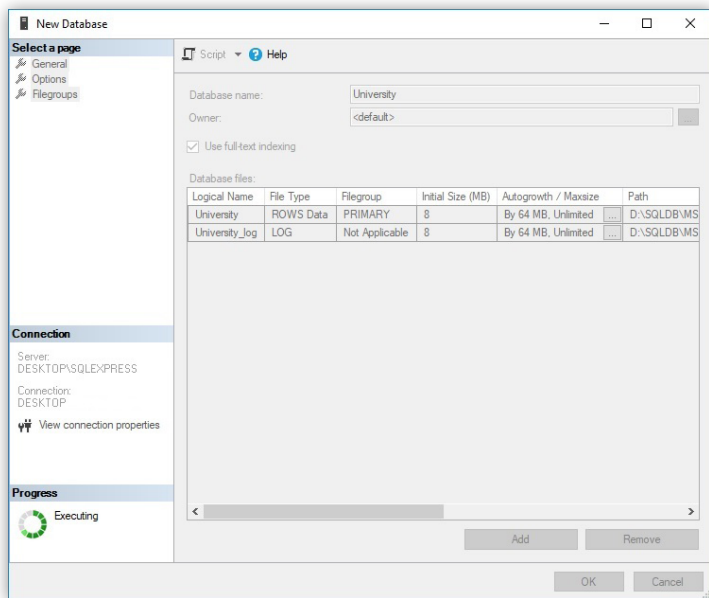


Рисунок 2.36. Процесс создания базы данных

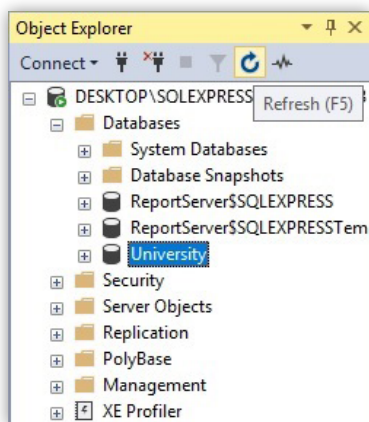


Рисунок 2.37. Результат создания новой базы данных

После закрытия окна, ваша база данных появится в списке **Databases** окна **Object Explorer** (Рисунок 2.37).

Если по какой-то причине, созданная вами база данных не отобразилась в списке, нажмите кнопку **Refresh** (Рисунок 2.37) и список баз данных обновится.

Переименование базы данных

После того как база данных создана, существует возможность внесения в нее ряда изменений, например ее можно переименовать. Для этого необходимо щелкнуть правой клавишей мыши на названии редактируемой базы данных и в контекстном меню выбрать пункт **Rename**, а затем ввести новое имя (Рисунок 2.38).

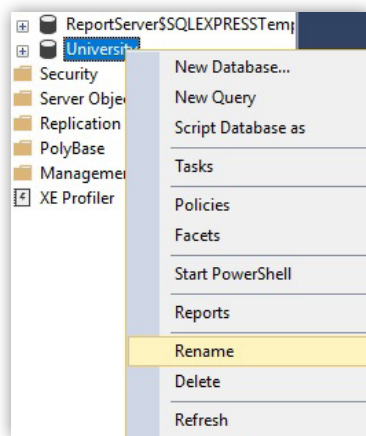


Рисунок 2.38. Переименование базы данных

Удаление базы данных

Существует также возможность удаления существующей базы данных, для этого необходимо вызвать контекстное меню на названии конкретной базы данных и выбрать пункт **Delete** (Рисунок 2.39).

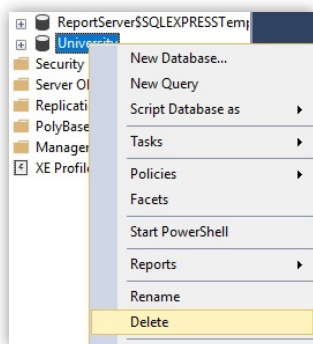


Рисунок 2.39. Начало удаления базы данных

После этого появится окно **Delete Object**, в котором после нажатия кнопки **ОК** вы увидите индикатор процесса удаления (Рисунок 2.40).

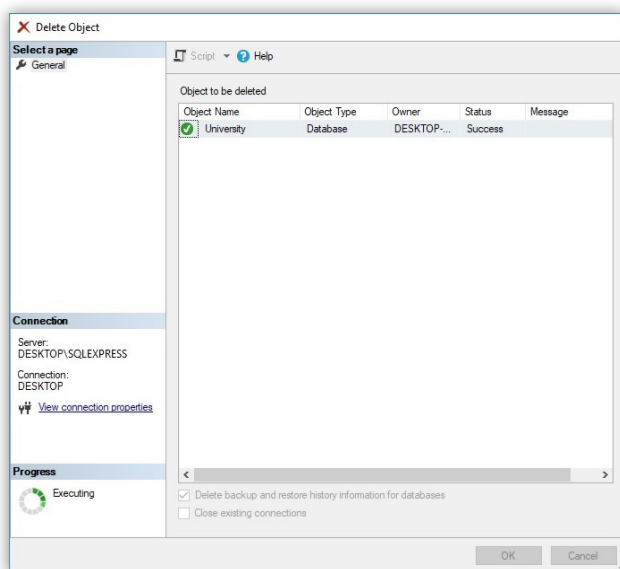


Рисунок 2.40. Завершение удаления базы данных

После того как окно закроется, ваша база данных исчезнет из списка **Databases** окна **Object Explorer**.

Домашнее задание

1. Установить Microsoft SQL Server 2016 Express и SQL Server Management Studio на свой компьютер.
2. Создать собственную базу данных. Задать: файлу данных начальный размер 15 MB, приращение размера файла 15% и неограниченный максимальный размер; файлу транзакций — начальный размер 10 MB, автоматическое приращение размера в 5 MB и максимальный размер в 60 MB.

Unit 2.
Таблицы,
типы данных,
запросы, основы
взаимодействия
с MS SQL Server

1. Таблицы

На прошлом занятии вы изучили порядок создания базы данных и выполнение различных действий по ее настройке. Однако сама по себе база данных особой пользы не принесет, для того чтобы ее можно было использовать в полном объеме необходимо наличие в ней ряда таблиц.

Таблицы в базе данных представляют собой множество данных, объединенных по определенному принципу. Каждая таблица описывает некую сущность при этом в столбцах хранится конкретное свойство сущности, а в строках — совокупность свойств для конкретной сущности.

Что бы было более понятно, разберем это на примере. Допустим, у нас есть таблица **Students**, которая хранит информацию о студентах некоего учебного заведения. В этой таблице столбцы **LastName**, **FirstName**, **BirthDate**, **Grants**, **Email** необходимы для хранения информации о фамилии, имени, дате рождения, стипендии и адресе электронной почты соответственно, а каждая строка в этой таблице содержит информацию о конкретном студенте. Например, студент Joshua Johnson родился 23 мая 1997 года, адрес его электронной почты jo@net.eu, и данные о его стипендии отсутствуют (Рисунок 1.1).

Id	LastName	FirstName	BirthDate	Grants	Email
1	Doe	John	1998-02-11	NULL	jh@net.eu
2	Jones	Jack	1997-11-05	1256.00	jj@net.eu
4	Johnson	Joshua	1997-05-23	NULL	jo@net.eu

Рисунок 1.1. Таблица с информацией о студентах

Теперь, когда предназначение таблиц стало понятным, мы можем приступить к процессу создания собственных таблиц. Для того чтобы создать таблицу в базе данных необходимо раскрыть пункт необходимой БД (в нашем случае **University**) в окне Object Explorer, на папке Tables нажать правой клавишей мыши и в контекстном меню выбрать пункт **Table...** (Рисунок 1.2).

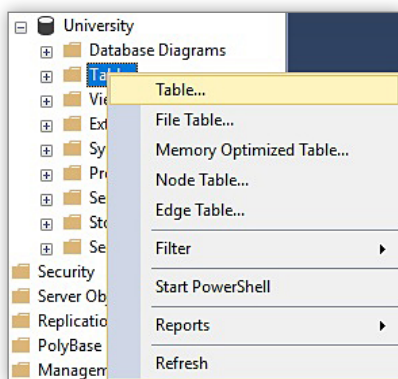


Рисунок 1.2. Создание новой таблицы (начало)

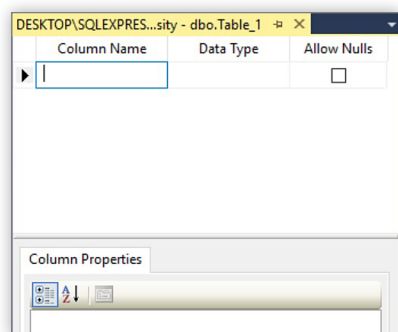


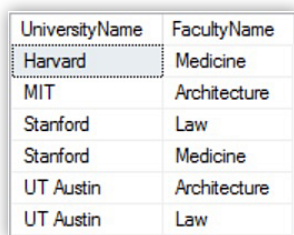
Рисунок 1.3. Создание новой таблицы (продолжение)

В открывшейся вкладке находится таблица, состоящая из трех столбцов **Column Name**, **Data Type** и **Allow**

Nulls. Эта таблица позволяет сформировать новую таблицу в текущей базе данных, путем создания необходимых полей, задав им значения в строках, соответственно, название (**Column Name**), тип данных (**Data Type**) и возможность использования неопределенного (**NULL**) значения (**Allow Nulls**) (Рисунок 1.3).

Первичный ключ

В примере прошлого раздела (Рисунок 1.1) мы намеренно проигнорировали столбец **Id** дело в том, что этот столбец не совсем обычный. В каждой таблице базы данных должно быть поле или совокупность полей, которое имеет общее название — первичный ключ. Значения первичного ключа никогда не должны повторяться, тем самым обеспечивая уникальность любой записи в таблице. В примере на рисунке 1.1 поле **Id** является первичным ключом таблицы **Students**.



UniversityName	FacultyName
Harvard	Medicine
MIT	Architecture
Stanford	Law
Stanford	Medicine
UT Austin	Architecture
UT Austin	Law

Рисунок 1.4. Пример составного первичного ключа

Существует также понятие составного первичного ключа, в этом случае первичный ключ состоит из двух и более полей, сочетание значений в которых должно быть уникальным. Например, в качестве составного первичного ключа можно использовать поля **UniversityName**

и **FacultyName** (Рисунок 1.4), что естественно, так как названия факультетов в одном университете не могут повторяться, а в различных университетах могут.

Из вышесказанного следует, что в любой таблице необходимо создать поле, которое будет являться первичным ключом. Обычно такое поле называют **Id** или в его названии присутствует слово **Id**, например, **StudId** и также обычно, но не обязательно, тип этого поля должен быть целочисленным (о типах данных будет рассказано далее).

Для того чтобы создать в нашей таблице поле с названием **Id** типа **int**, необходимо в новой строке таблицы в поле **Column Name** ввести соответствующее название, а в поле **Data Type** из выпадающего списка выбрать название требуемого типа данных (Рисунок 1.5).

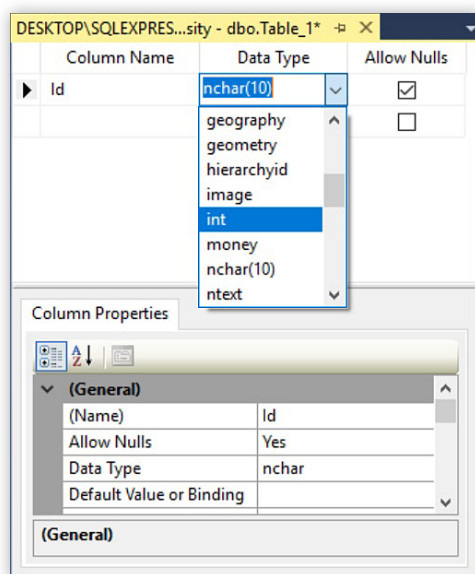


Рисунок 1.5. Выбор требуемого типа данных

Для того чтобы сделать это поле первичным ключом необходимо нажать на него правой кнопкой мыши и в появившемся контекстном меню выбрать пункт **Set Primary Key** (Рисунок 1.6).

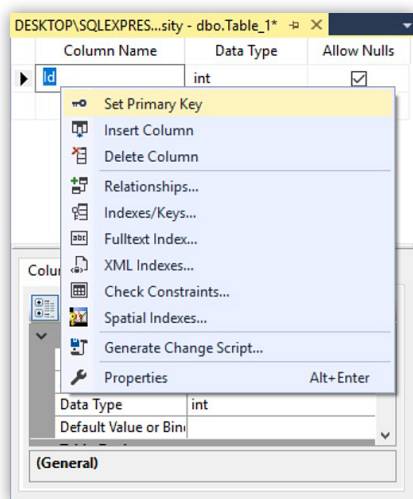


Рисунок 1.6. Создание первичного ключа

В качестве визуального результата вы заметите изображение ключа рядом с выбранным полем, также вы заметите невозможность установки первичному ключу неопределенного значения (сброшено значение в поле **Allow Nulls**).

Существует возможность настроить получение уникального значения первичного ключа путем задания автоматического инкрементирования значений в требуемом поле. Для этого необходимо в таблице **Column Properties** раскрыть свойство **Identity Specification** и в выпадающем списке поля (**Is Identity**) выбрать значение **Yes** (Рисунок 1.7).

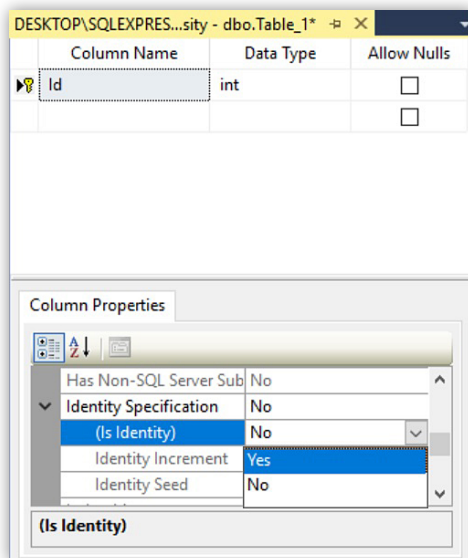


Рисунок 1.7. Настройка уникальности первичного ключа

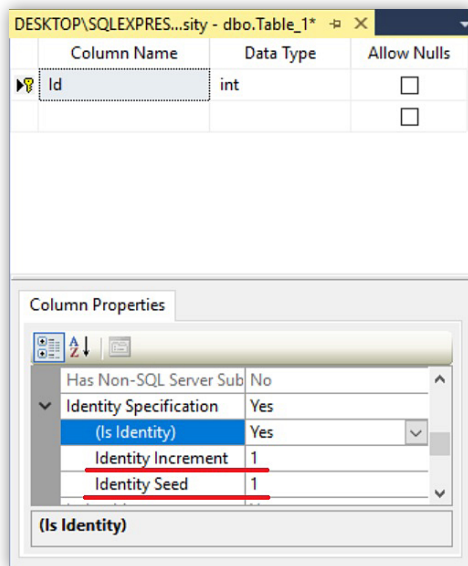


Рисунок 1.8. Настройка автоматического инкрементирования значений

После этого станут доступны поля **Identity Increment** и **Identity Seed**. Изменив значение в поле **Identity Increment**, вы можете задать величину приращения первичного ключа, при помощи поля **Identity Seed** можно задать начальное значение соответствующего поля таблицы, по умолчанию значения в этих полях равны 1 (Рисунок 1.8).

Первичные ключи также используются для создания связей между таблицами, но об этом вы более подробно узнаете в четвертом уроке.

Значение по умолчанию

При создании таблицы существует возможность задать значения по умолчанию для конкретных полей, то есть в том случае если при добавлении новой записи в таблицу полю не будет указано конкретное значение, то будет записано указанное значение по умолчанию.

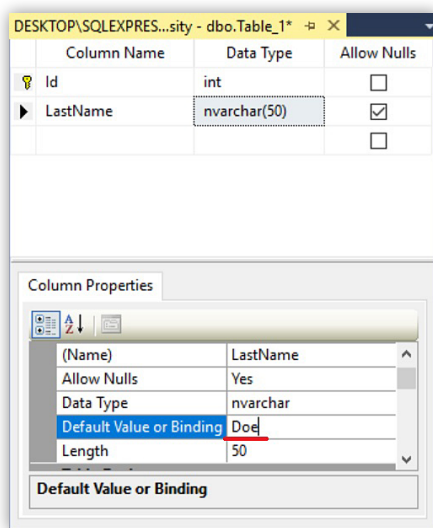


Рисунок 1.9. Настройка значения по умолчанию

Продолжим формирование полей нашей таблицы. Следующее поле имеет название **LastName**, оно предназначено для хранения фамилии студента и имеет тип данных **nvarchar(50)**.

Для того чтобы задать полю значение по умолчанию необходимо в таблице **Column Properties** задать требуемое значение свойству **Default Value or Binding**, в нашем случае — **Doe** (Рисунок 1.9).

Уникальность

При создании таблицы может возникнуть необходимость в использовании полей с уникальными значениями для каждой записи и **Management Studio** предоставляет такую возможность.

В нашем случае таким полем является поле для хранения электронного адреса студента (**Email**). Для того чтобы объявить это поле уникальным, необходимо нажать по нему правой клавишей мыши и в контекстном меню выбрать пункт **Indexes/Keys...** (Рисунок 1.10).

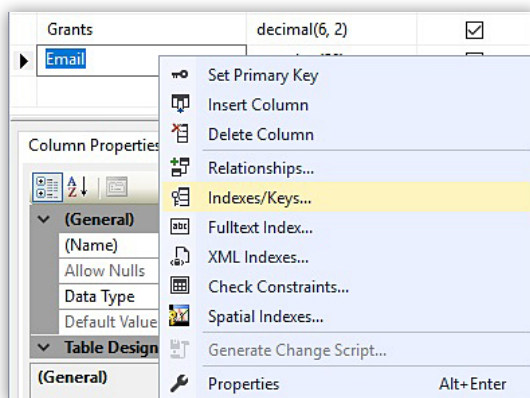


Рисунок 1.10. Создание уникального ключа (начало)

В открывшемся окне **Indexes/Keys** нужно добавить настраиваемый ключ, нажав на кнопку **Add**, при этом система сгенерирует произвольное имя этому ключу, изменить которое можно в поле **(Name)** раздела **Identity** (Рисунок 1.11).

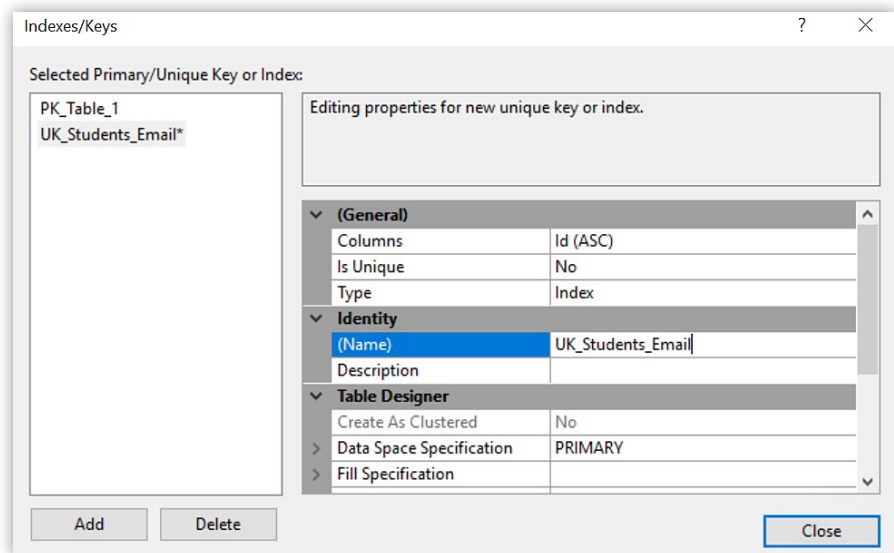


Рисунок 1.11. Изменение имени ключа

Следующее что необходимо сделать — это указать имя настраиваемого столбца в поле **Columns**, для чего нужно в этом поле нажать кнопку с тремя точками и в выпадающем списке появившегося окна **Index Columns** выбрать требуемое название, после чего нажать кнопку **ОК** (Рисунок 1.12).

Последним шагом в настройке уникального поля является указание типа ключа в поле **Type** раздела **(General)**, в выпадающем списке которого необходимо выбрать значение **Unique Key** (Рисунок 1.13).

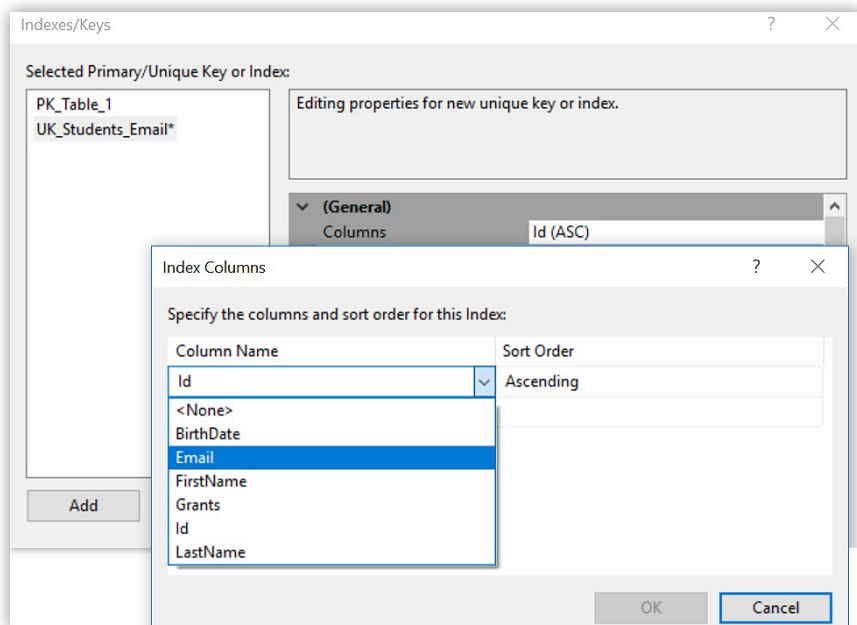


Рисунок 1.12. Выбор настраиваемого столбца

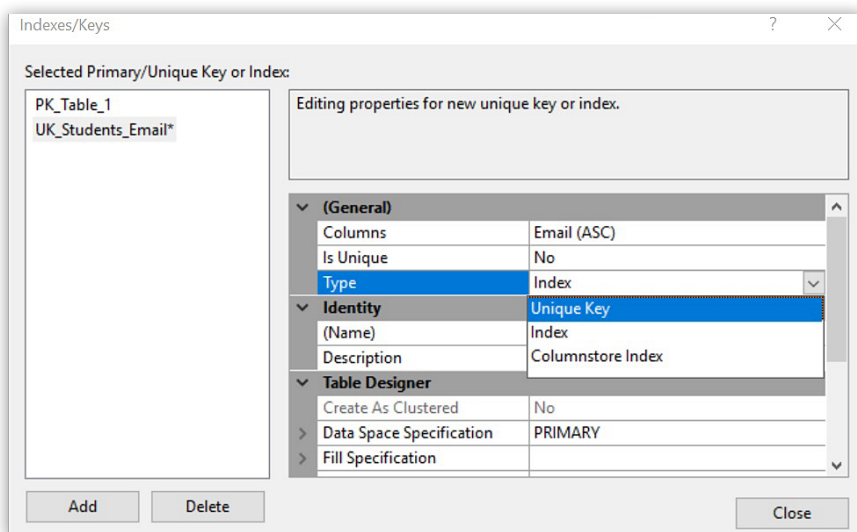


Рисунок 1.13. Указание типа ключа

Чтобы настройки уникальности поля вступили в силу нужно закрыть окно **Indexes/Keys**, нажав кнопку **Close**.

После того как вы создали и настроили все поля таблицы имеет смысл изменить имя таблицы, заданное по умолчанию. Это можно сделать в окне свойств текущей таблицы, указав в поле **(Name)** раздела **Identity** требуемое имя таблицы (Рисунок 1.14).

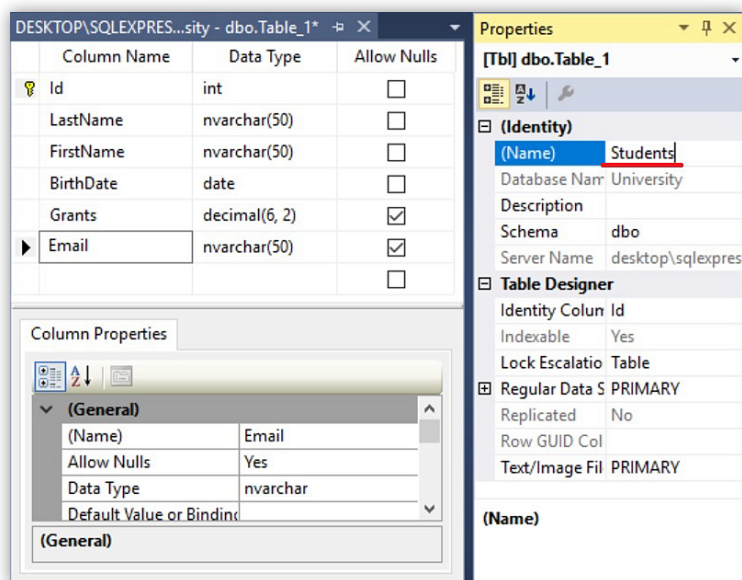


Рисунок 1.14. Переименование таблицы

Сохранение созданной нами таблицы происходит стандартным образом — нажатием кнопки с изображенной дискетой на панели инструментов или сочетанием клавиш **Ctrl+S**. После этого таблица **Students** появится в списке таблиц базы данных **University**.

Для того чтобы убедиться в правильности выполненных нами настроек, заполним таблицу **Students** зна-

чениями, для этого необходимо нажать правой клавишей мыши на имени таблицы и в контекстном меню выбрать пункт **Edit Top 200 Rows** (Рисунок 1.15).

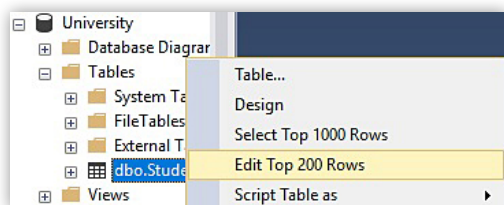


Рисунок 1.15. Заполнение таблицы значениями (начало)

В появившейся вкладке вы увидите таблицу с **NULL**-значениями в каждом поле, заполним их определенными значениями.

Вы можете заметить, что при добавлении в таблицу очередной записи значение в поле **Id** генерируется автоматически, что происходит благодаря настройке автоинкрементирования первичного ключа.

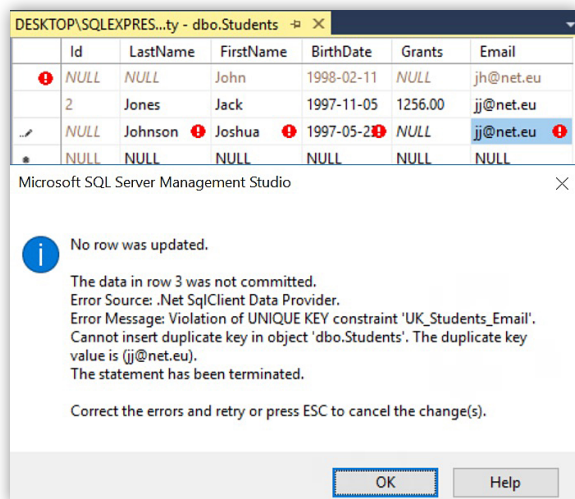


Рисунок 1.16. Ошибка: нарушение уникальности записи

При попытке указать в поле **Email** повторяющееся значение, вы увидите ошибку, в которой сообщается о нарушении уникальности и невозможности дублирования конкретной записи (Рисунок 1.16).

В первой записи мы не указали значение для поля **LastName** (проверка на значение по умолчанию) и **Management Studio** реагирует на это как на ошибку, правда в описании указывается, что ошибке нет, и эта проблема исчезнет после сохранения таблицы (Рисунок 1.17).

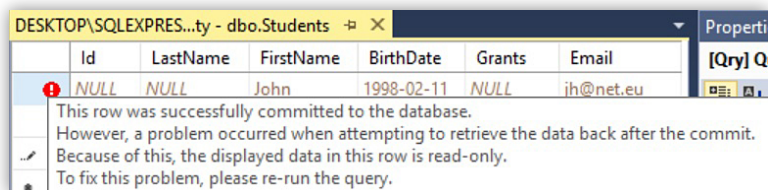


Рисунок 1.17. Ошибка: незаполненное поле

Для того чтобы проверить правильность заполнения таблицы **Students** значениями необходимо нажать правой клавишей мыши на имя таблицы и выбрать пункт **Select Top 1000 Rows** (Рисунок 1.18).

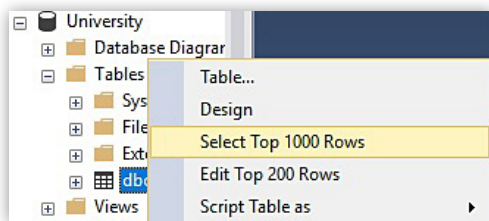


Рисунок 1.18. Выбор записей в таблице (начало)

В открывшейся вкладке вы увидите текст запроса, а в результатах — полученную таблицу с названиями полей и данными в них (Рисунок 1.19).

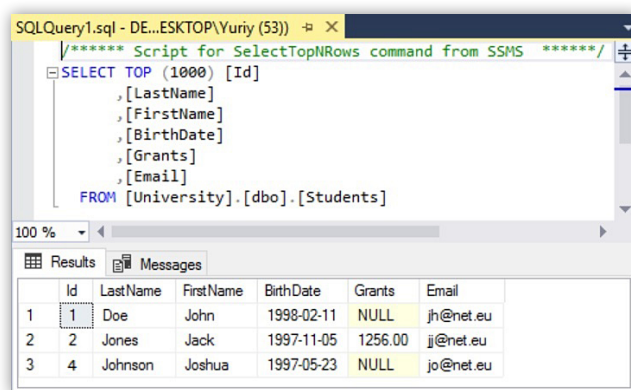


Рисунок 1.19. Результат выбора записей таблицы

И хотя текст запроса вам пока непонятен, вы можете убедиться, что все записи заполнились корректно.

2. Типы данных

Как вы уже заметили, для хранения информации в таблицах используются различные типы данных, рассмотрим основные из них.

Целочисленные типы данных

Целочисленные типы данных используются для хранения точных числовых данных, например, возраста человека или количества единиц определенного товара. Ниже представлена таблица целочисленных типов данных с указанием названия, диапазона допустимых значений и количества занимаемой памяти (Таблица 2.1).

Таблица 2.1. Целочисленные типы данных

Название	Диапазон	Память
bigint	от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807	8 байт
int	от -2 147 483 648 до 2 147 483 647	4 байта
smallint	от -32 768 до 32 767	2 байта
tinyint	от 0 до 255	1 байт

Еще одним целочисленным типом данных является **bit**, он занимает один байт памяти и может хранить значения 0, 1 или **NULL**, фактически этот тип данных является аналогом логического типа данных в таких языках программирования как C++ или C#.

Типы данных для хранения текста

Для хранения текстовой информации в базе данных используются четыре основных типа данных:

- `char(n)` — предназначен для хранения строк фиксированной длины не в кодировке Unicode, где вместо `n` указывается возможное количество символов в строке от 1 до 8000, память при этом выделяется по одному байту на символ;
- `varchar(n | max)` — также предназначен для хранения строк не в кодировке Unicode, но переменной длины, допустимое количество символов в строке можно задать либо указав значение от 1 до 8000, либо использовать для этого слово `max` и в этом случае для хранения строки выделится память до 2 ГБ;
- `nchar(n)` — предназначен для хранения строк фиксированной длины в кодировке Unicode, при помощи `n` задается максимально возможное количество символов в строке от 1 до 4000, размер выделенной памяти по два байта на символ;
- `nvarchar(n | max)` — тип данных, который предназначен для хранения строк переменной длины в кодировке Unicode, максимальное количество символов в строке можно задать от 1 до 4000 или выделить память для хранения строки до 2 ГБ, если использовать ключевое слово `max`.

В чем же состоит разница между строкой переменной длины и фиксированной строкой?

Если, например, для хранения строки указан тип данных `char` или `nchar` с размером 10 символов (`char(10)` или `nchar(10)`), а сохраняется строка из шести символов, то количество выделенной памяти будет соответствовать заявленному размеру — 10 или 20 байт, соответственно.

Если для хранения строки указан тип данных `varchar` или `nvarchar` с размером 10 символов (`varchar(10)` или `nvarchar(10)`), а сохраняется строка из шести символов, то размер выделенной памяти будет соответствовать реальному количеству символов — 6 или 12 байт, соответственно.

Вы также можете встретить типы данных `text` и `ntext`, но эти типы данных являются устаревшими и их использование не рекомендовано.

Вещественные типы данных

Для хранения числовых типов данных с плавающей точкой используются вещественные типы данных `float` и `real`.

При помощи типа данных `float` можно хранить числа в диапазоне от $-1,79E+308$ до $-2,23E-308$, 0 и от $2,23E-308$ до $1,79E+308$.

При использовании типа данных `float(n)`, указывая значение `n`, вы можете устанавливать точность числа и естественно изменять размер выделяемой памяти. Если значение `n` с 1 по 24, то SQL Server трактует его как 24 и выделяет память в 4 байта для хранения числа с точность до 7 знаков. Если в `n` указывается значение с 25 по 53, то SQL Server трактует его как 53 и выделяет 8 байт памяти для хранения числа с точность до 15 знаков. По умолчанию значение `n` равно 53.

Типа данных `real` позволяет хранить числа в диапазоне от $-3,40E+38$ до $-1,18E-38$, 0 и от $1,18E-38$ до $3,40E+38$.

Синонимом для типа данных `real` является `float(24)`.

Типы данных для хранения даты и времени

Для хранения в таблицах даты и времени SQL Server предоставляет ряд типов данных, которыми вы можете воспользоваться в зависимости от поставленной задачи.

Тип данных `datetime` позволяет хранить дату и время в 24-часовом формате с указанием долей секунды, в диапазоне с 1 января 1753 года по 31 декабря 9999 года, например, `2017-12-28 15:20:35.693`.

Тип данных `datetime2` позволяет хранить дату и время в 24-часовом формате, но имеет большую точность долей секунды по сравнению с типом `datetime` и диапазон дат с 1 января 0001 года по 31 декабря 9999 года, например, `2017-12-28 15:20:35.6930000`.

Тип данных `datetimeoffset` позволяет хранить дату и время в 24-часовом формате с указанием долей секунды с учетом часового пояса в диапазоне с 1 января 0001 года по 31 декабря 9999 года, например, `2017-12-28 15:20:35.6930000 +00:00`.

Предыдущие типы данных выводят довольно точные значения времени, но, если вам достаточно получить только дату и время, например, время покупки товара, тогда вы можете использовать тип данных `smalldatetime`. Этот тип данных позволяет хранить дату и время в 24-часовом формате с секундами, всегда равными нулю, без долей секунды в диапазоне с 1 января 1900 года по 6 июня 2079 года, например, `2017-12-28 15:20:00`.

Если поле вашей таблицы предназначено для хранения только даты, например, даты рождения человека, то вам больше подойдет тип данных `date`, который позво-

ляет хранить дату в диапазоне с 1 января 0001 года по 31 декабря 9999 года, например, **2017-12-28**.

Тип данных **time** позволяет хранить только время в 24-часовом формате с указанием долей секунды без учета часового пояса в диапазоне от 00:00:00.00000000 до 23:59:59.9999999, например, **15:20:35.6930000**.

Типы данных с фиксированной точкой

Для хранения вещественных значений в более точном формате используются типы данных **decimal(p, s)** и **numeric(p, s)**. Эти типы данных являются взаимозаменяемыми и позволяют хранить данные в диапазоне от $-10^{38}+1$ до $10^{38}-1$.

При помощи параметра **p** задается общее количество цифр в числе, как целой, так и дробной части. Диапазон значений от 1 до 38 (по умолчанию 18), чем большее значение вы укажете, тем больше байт памяти будет выделено для хранения информации. Значение в параметре **s** определяет количество цифр дробной части числа в диапазоне от 0 до **p** (по умолчанию 0).

Тип **decimal(6,2)** мы использовали в качестве типа данных для поля **Grants** таблицы **Students** из раздела № 1. В этом случае общее количество цифр в числе равно 6, а цифр после запятой 2, то есть целая часть должна состоять из 4 цифр.

Другие типы данных

Для хранения денежных значений используются типы данных **money** и **smallmoney**, которые представляют собой вещественные числа, дробная часть которых

предназначена для хранения мелочи (копейки, центы и т.д.) (Таблица 2.2).

Таблица 2.2. Денежные типы данных

Название	Диапазон	Память
<code>money</code>	от -922 337 203 685 477,5808 до 922 337 203 685 477,5807	8 байт
<code>smallmoney</code>	от -214 748,3648 до 214 748,3647	4 байта

Для хранения различной информации в бинарном виде используются типы данных `binary(n)` и `varbinary(n | max)`. Параметр `n` может принимать значения от 1 до 8000, если указать ключевое слово `max`, то выделится память для хранения 2 ГБ информации.

Если у вас есть необходимость хранить данные в двумерной системе координат, тогда вы можете использовать тип данных `geometry`.

Для хранения координат широты и долготы объекта предназначен тип данных `geography`.

Хранение XML-документов обеспечивает тип данных `xml`.

Временный результирующий набор данных, полученных в результате запроса, сохраняется в типе данных `table`.

Обычно при создании таблицы полям указываются такие типы данных, максимальный размер которых гарантировано, позволит сохранить требуемый диапазон значений, такой подход используется в целях уменьшения размера базы данных.

3. Индекс

Что такое индекс?

Индекс — это физическая структура данных в БД, при помощи которой осуществляется ускоренный доступ к необходимой информации, использование подходящего индекса значительно улучшает производительность запроса.

Для того чтобы лучше понять, что такое индекс, представим ситуацию, когда вам необходимо найти определенную главу в любой книге. Существуют два способа как это сделать — последовательный перебор всех страниц либо получение номера нужной страницы из оглавления книги. Конечно же, используя второй способ, вы быстрее достигните требуемого результата. Так вот индексы, также как оглавление в книге, позволяют ускорить доступ к требуемой информации в базе данных, если подходящий индекс отсутствует, то поиск осуществляется последовательным перебором всех записей таблицы.

Цели и задачи индекса

К этому моменту вы можете сказать: «Давайте назначим индексы для всех полей таблицы, что может быть проще?» Однако такой подход принесет больше вреда, чем пользы, ведь индекс по сути представляет собой копию данных того поля, для которого он создан, следовательно, при создании индекса увеличивается размер базы данных. Также наличие большого количества индексов увеличивает время выполнения операций по измене-

нию данных (**INSERT**, **UPDATE**, **DELETE**), потому что изменение полей автоматически приводит к изменению индексов, в результате чего происходит фрагментация индекса. Поэтому создавать индексы необходимо с осторожностью и помнить о некоторых особенностях:

- индексы автоматически создаются для уникальных полей таблицы и полей, которые указаны в качестве первичных ключей;
- индексы следует создавать для полей таблицы, по которым часто производится поиск (можно также создавать один индекс для набора столбцов);
- для создания индекса наиболее подходят те поля таблицы, у которых количество повторяющихся значений минимально;
- индексы также можно создавать и для представлений (**VIEWS**), которые будут рассмотрены в курсе MS SQL Server.

Внутреннее устройство индекса

Прежде чем мы опишем внутреннее устройство индекса необходимо разобраться в способе хранения данных в самой базе данных.

Минимальной единицей распределения памяти в базе данных является страницы, размер каждой страницы 8 КБ. Существует два вида страниц — страницы данных и страницы индексов. Восемь страниц образуют экстенд, который используется для эффективного управления страницами, размер экстенда, соответственно, 64 КБ.

Для хранения индексов используется структура данных в виде сбалансированного дерева В-дерево (*Balanced*

Tree — B-Tree). Такая структура представляется в виде дерева, ветви которого направлены вниз, и обеспечивает автоматическую сбалансированность узлов, то есть количество ветвей справа от корневого узла приблизительно равно количеству ветвей слева. Такой способ хранения обеспечивает простой и быстрый способ выборки необходимой информации (Рисунок 3.1).

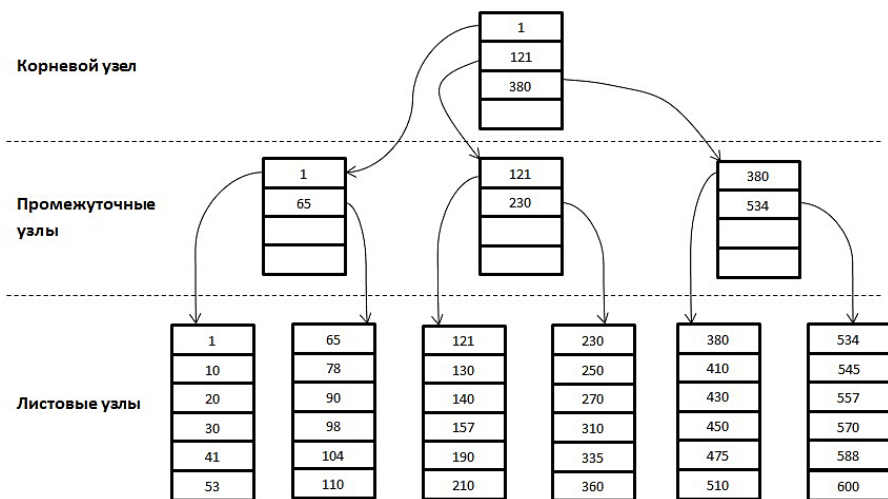


Рисунок 3.1. Пример B-дерева

Индексы бывают кластерные и некластерные.

Кластерный индекс отличается тем, что на его листовом уровне содержатся действительные данные, то есть после достижения листового уровня в таком индексе система получает требуемые данные. В таблице может быть только один кластерный индекс и создается автоматически на поле, которое указано в качестве первичного ключа.

На листовом уровне некластерного индекса находится либо так называемый идентификатор строки (*Row*

ID — *RID*), в котором содержится информация о требуемой записи в виде номера: экстенста, страницы и смещении строки на странице, либо кластеризованный ключ, который содержит информацию о кластеризованном индексе. После получения этой информации осуществляется дальнейший поиск данных. Количество некластерных индексов для одной таблицы неограниченно.

Использование индексов будет подробно рассмотрено вами в курсе MS SQL Server.

4. Системные базы данных и таблицы

При установке SQL Server 2016 автоматически создаются четыре системных базы данных, наличие которых необходимо для корректного функционирования SQL Server. Вы можете их увидеть, если в окне Object Explorer Management Studio развернете папку Databases, а затем подпапку System Databases (Рисунок 4.1).

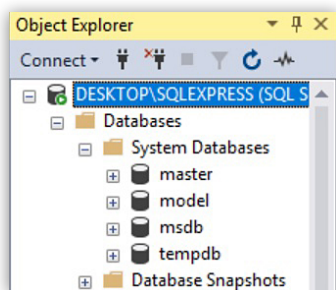


Рисунок 4.1. Системные базы данных

В этой папке находятся следующие системные базы данных:

- **master** — содержит всю информацию о самом SQL Server, а также обо всех базах данных, без этой базы данных запуск SQL Server не возможен;
- **model** — служит шаблоном при создании всех пользовательских баз данных, наличие этой базы данных обязательно;
- **msdb** — эта база данных предназначена для создания расписания заданий, например, создания резервных копий и восстановления любой базы данных;

- **tempdb** — используется для хранения различных временных объектов.

Помимо системных БД также создаются и системные таблицы, которые являются общими для всех баз данных. Системных таблиц довольно много поэтому мы рассмотрим только некоторые из них, за более подробной информацией рекомендуем обратиться к справочной документации:

- **backupfile** — содержит информацию обо всех файлах баз данных на момент создания резервных копий, находится в БД msdb;
- **restorefile** — содержит информацию по каждому восстановленному файлу БД, расположена в базе данных msdb;
- **log_shipping_primary_secondaries** — содержит информацию, которая связывает БД-источник с БД-получателем, находится в базе данных msdb;
- **cdc.lsn_time_mapping** — в этой таблице хранятся данные о всех транзакциях из таблицы изменений;
- **MSdbms** — в этой таблице находится полный список СУБД, отличных от MS SQL Server, которые поддерживают совместимую с MS SQL Server репликацию баз данных, находится в БД msdb;
- **MSreplication_options** — содержит данные, используемые при репликациях, расположена в базе данных master;
- **sys.sysoledbusers** — содержит данные по всем пользователям текущего сервера, находится в базе данных master.

5. Запросы

Введение в язык структурированных запросов SQL

Основным предназначением любой базы данных является накопление требуемой информации и предоставление ее при необходимости. Необходимые данные мы можем получить, запросив их у БД, то есть, написав некоторый код — запрос.

Для того чтобы написать соответствующий запрос нужен специализированный язык программирования. Традиционные языки программирования, существовавшие на момент появления реляционной модели данных, такие как COBOL или Fortran, на эту роль не подошли. Поэтому был разработан новый язык программирования — **SQL** (*Structured Query Language* — язык структурированных запросов).

Язык SQL

Первая версия языка реляционных баз данных появилась в начале 70-х годов прошлого столетия и называлась **SEQUEL** (*Structured English Query Language* — язык структурированных запросов на основе английского). Этот язык был разработан компанией IBM как часть проекта System/R, направленного на реализацию реляционной СУБД. Через несколько лет была выпущена вторая версия — **SEQUEL/2**, позже язык был переименован в **SQL**.

Язык SQL является декларативным языком, при помощи которого вы обращаетесь к базе данных, запра-

шивая требуемую информацию, то есть вы указываете что вам необходимо, а задачу как это получить решает сама СУБД.

Таким образом, основное предназначение языка SQL — обеспечение взаимодействия с базами данных, путем формирования различного рода запросов, которые состоят из специальных операторов. Операторы — это инструкции, при помощи которых вы запрашиваете информацию из БД, указывая какие данные вас интересуют, откуда их нужно получить и, при необходимости, ограничивая полученный результат. Например, запрос может звучать следующим образом: «Покажите мне фамилии только тех студентов, которые родились в ноябре». В данном случае часть запроса «Покажите мне» указывает, что требуется предоставить, следующая часть запроса «студентов» — информирует откуда взять данные, условие «родились в ноябре» ограничивает количество записей.

В виде SQL-запроса это выглядит следующим образом:

```
SELECT LastName  
FROM Students  
WHERE MONTH(BirthDate) = 11;
```

Не вдаваясь в подробности выполнения запроса, о которых вы узнаете из следующего урока, можно внести некоторую ясность. При помощи оператора **SELECT** вы прописываете **что** вам нужно, в операторе **FROM** вы указываете **откуда** необходимо взять требуемую информацию, а в операторе **WHERE** находится **условие**, которое конкретизирует конечный результат.

Стандарты языка SQL

Несмотря на то, что язык SQL был принят в качестве основного языка по работе с базами данных, существование нескольких производителей СУБД привело к появлению нескольких реализаций (диалектов) языка SQL от разных производителей. Такое положение вещей исключало возможность переноса программного обеспечения с одной СУБД на другую, поэтому стал вопрос о стандартизации языка SQL.

В 1986 году *Американский национальный институт стандартов (ANSI)* разработал первый стандарт языка SQL, который был утвержден *Международной организацией стандартов (ISO)* в 1987 году, получивший название SQL-86. После внесения изменения в оригинальный стандарт в 1989 году вышел следующий стандарт языка — SQL-89.

Последующие расширения языка SQL привели к появлению нескольких стандартов в 1992, 1999, 2003, 2006, 2008, 2011 и 2016 годах. В настоящее время действует, соответственно стандарт SQL-2016.

Диалекты языка SQL

Несмотря на наличие большого количества стандартов в настоящее время не существует единого диалекта языка SQL. Введение производителями СУБД новых функциональных средств вносит в существующие диалекты новые изменения, тем самым диалекты все больше отличаются друг от друга.

Приведем список диалектов языка SQL в СУБД наиболее известных производителей:

- **T-SQL** (*Transact-SQL*) — этот диалект используется в СУБД Microsoft SQL Server и Sybase ASE;
- **PL/SQL** (*Procedural Language/SQL*) — данный диалект используется в СУБД Oracle;
- **SQL/PSM** (*SQL/Persistent Stored Module*) — этот диалект используется в СУБД MySQL;
- **PLpg/SQL** (*Procedural Language/postgreSQL*) — диалект реализован в СУБД PostgreSQL;
- **SQLPL** (*SQL Procedural Language*) — этот диалект реализован в СУБД DB2;
- **Jet SQL** — данный диалект реализован в СУБД Microsoft Access.

Диалект Transact-SQL

Transact-SQL — реализация языка SQL, разработанная корпорацией Microsoft для СУБД Microsoft SQL Server. Знакомиться с этим диалектом вы будете на протяжении текущего и последующего курсов, а пока приведем его краткое описание.

T-SQL позволяет использовать различные операторы: арифметические (+, −, *, /, %), логические (AND, OR, NOT), сравнения (=, >, <, >=, <=, <>) и операторы для работы со множествами (IN). При использовании T-SQL существует возможность создавать переменные при помощи команды DECLARE, используя для этого специальные символы — идентификаторы (@). В T-SQL содержится условный оператор (IF) и цикл (WHILE). При написании запросов существует возможность вызова встроенных функций (COUNT, SUM, MIN, MAX, DATEDIFF, ABS

и т.д.). Для комментирования кода в T-SQL используется либо строчный (--), либо блочный (/* */) комментарий.

Понятия DDL, DML, DCL

SQL-операторы делятся на три категории: операторы **DDL** (*Data Definition Language* — язык описания данных), **DML** (*Data Manipulation Language* — язык управления данными) и **DCL** (*Data Control Language* — язык управления доступом к данным). Рассмотрим эти операторы более подробно.

Операторы DDL позволяют работать со структурой данных в БД:

- создание объекта (**CREATE**);
- изменение объекта (**ALTER**);
- удаление объекта (**DROP**).

Операторы DML используются при работе с данными в БД:

- запрос определенной информации (**SELECT**);
- вставка необходимых данных в таблицу (**INSERT**);
- обновление существующих данных в таблицы (**UPDATE**);
- удаление данных из таблицы (**DELETE**).

Операторы DCL позволяют управлять доступом к базе данных:

- предоставление доступа пользователя для работы с объектом (**GRANT**);
- запрет на доступ пользователя к объекту (**DENY**);
- отмена привилегий доступа пользователя к объекту (**REVOKE**).

Домашнее задание

3. Создайте базу данных, которая содержит таблицу с информацией о книгах. Названия необходимых полей продумайте самостоятельно. Используйте все знания, полученные вами к этому моменту.